UMEÅ UNIVERSITY **2021**
Department of Computing Science
Bachelor Thesis

Investigating the effect of implementing Data-Oriented Design principles on performance and cache utilization

Frank Nyberg

Supervisor
Jerry Eriksson

Abstract

Game engines process a lot of data under strict deadlines. Therefore, measures to increase performance are important in this area. Data-Oriented Design (DOD) promotes principles that are meant to increase performance by better cache utilization. The purpose of this thesis is to examine a selection of these principles to give a better understanding of how DOD affects CPU time and the rate of cache misses, with focus on the area of game development. More specifically, the principles examined are removal of run-time polymorphism, iteration over contiguous data, and lowering the amount of data in hot loops. Also, the Entity-Component-System pattern is examined, which is based upon DOD principles. The approach was to first present a theoretical background on the subject, and then to conduct tests by implementing a simulation of movement and collision detection utilizing said principles. The tests were written in C++ and executed on an Intel Core i7 4770k with no rendering. CPU time was measured in updated entities per µs, and cache utilization was measured in the form of cache miss rate. The results showed that the DOD principles did increase performance. Cache miss rate was also lower, with the exception of when removing run-time polymorphism. The conclusion is that Data-Oriented Design, used in game development, is likely to result in better performance, mostly as a result of better cache utilization.

Contents

1 Introduction **1**
- 1.1 Purpose and Research Questions . . . 1
- 1.2 Method . . . 2
- 1.3 Delimitations . . . 2

2 Background **4**
- 2.1 Hardware aspects . . . 4
  - 2.1.1 The CPU . . . 4
  - 2.1.2 Memory latency . . . 5
  - 2.1.3 The CPU cache . . . 6
  - 2.1.4 Cache operation . . . 6
  - 2.1.5 Associativity and its relation to performance . . . 8
  - 2.1.6 Prefetching . . . 10
- 2.2 Programming . . . 10
  - 2.2.1 Data locality . . . 11
  - 2.2.2 Structure size and alignment . . . 12
  - 2.2.3 Instruction locality and size . . . 13
  - 2.2.4 Run-time polymorphism . . . 14
  - 2.2.5 Game development and OOD . . . 15
  - 2.2.6 Data-Oriented Design . . . 16
  - 2.2.7 The Entity-Component-System pattern . . . 17

3 Related Work **20**

4 Method **22**
- 4.1 Collision detection algorithm . . . 23
- 4.2 Implementation details . . . 25
- 4.3 Test parameters and working set size . . . 33
- 4.4 Language and compiler . . . 34
- 4.5 Profiling . . . 35
- 4.6 Hardware . . . 36

5 Results and analysis **38**
- 5.1 Overview . . . 38
- 5.2 Removal of run-time polymorphism . . . 43
- 5.3 Contiguous allocation . . . 44
- 5.4 Reducing structure size . . . 46
- 5.5 Using an Entity-Component-System design . . . 47

6 Discussion **50**
- 6.1 Interpretation of results . . . 50
  - 6.1.1 Summary . . . 51
  - 6.1.2 Applicability in the area of game development . . . 51
- 6.2 Limitations . . . 52
- 6.3 Conclusion and recommendations . . . 53
- 6.4 Future Work . . . 54

References **55**

1 Introduction

In the world of computer games, large amounts of data are often processed. A game world, especially for some game types like real-time strategy games, might contain thousands of objects which must be animated, drawn, and processed according to the game's rules. This must happen many times each second so that the player does not feel like the game is sluggish or unresponsive. Therefore, measures to increase performance are especially important in this area.

Game engines are often structured in a certain way. They generally tend to make heavy use of features associated with an Object-Oriented Design (OOD) such as large inheritance hierarchies and polymorphic objects. There is, however, a trend in the video game industry towards an approach called the Data-Oriented Design (DOD), which battles many of the patterns which are visible in past and present game development. Iterating over a collection of polymorphic objects and running some common function such as `Update()` or `Move()` is a commonly seen example. This might give a lot of flexibility and modularity but is frowned upon by DOD proposers for the reason that it may harm performance. Apart from doing away with run-time polymorphism, DOD promotes a set of guidelines which are all centered around effective processing of data. The mechanisms by which this happens are many, such as easier and less costly parallelization and better use of enhanced instruction sets found in certain CPUs. The main mechanism, however, is said to be due to a more effective use of the CPU cache.

The CPU cache is (most often) a hierarchy of a few small and fast memories located on or near the CPU, which contain copies of the most recently and most frequently used data. The size is often in the kilobyte to megabyte range, which is tiny compared to the multiple gigabytes found in the main memory, but they are at the same time an order of magnitude faster, or even more than that. Making sure that the caches are used to their full potential can therefore yield a large performance increase.

1.1 Purpose and Research Questions

Previous work has shown that a Data-Oriented Design may result in better performance, both in terms of CPU time and frame rate, than an equivalent Object-Oriented Design. The number of cache misses has also been shown to be lower with such a design. The exact mechanisms behind the changes have not, however, been very well charted. As the DOD guidelines are numerous and may vary between descriptions, it is hard to attribute the change in performance to a specific parameter change. The premise has also been that OOD is the antithesis of DOD, but many sources claim that Data-Oriented principles can also be applied to an Object-Oriented Design. Therefore, there always exist the possibility that a comparison between the two only shows the difference between a ”bad” Object-Oriented Design, in terms of, for example, cache usage, to a ”good” Data-Oriented Design.

This report therefore aims to give a better understanding of the implications of implementing a Data-Oriented Design, by breaking the design down into its core principles and examining the ones claiming to have an effect on the cache. The main focus lies on the principles of removing run-time polymorphism, iterating over linear data rather than using pointers, and lessening the size of the data which is iterated upon. Finally, the effect of employing an Entity-Component-System (ECS) pattern is examined. This

is a pattern which is built upon DOD guidelines, and which deviates from the classical way of modeling entities in a game as objects. As such, it is often used to represent DOD in comparisons in the style of "DOD vs. OOD".

The report also intends to deliver a more fine-grained analysis on the effect on the individual cache levels, as previous works have mostly looked at the Last Level Cache.

The research question this thesis aims to answer is therefore: To what degree does restructuring code in accordance with Data-Oriented Design principles affect CPU time and cache utilization. And how is the result applicable in the field of video game development?

1.2 Method

To answer the research question, first a theoretical background is given on the hardware aspect of how memory access affects CPU performance. Then, an overview is given of how different coding practices may affect memory latency, and by implication, performance, and how practices differ between Data-Oriented Design and the more traditional way of programming games.

Finally, to examine the real-world impact of adapting the coding practices proposed by DOD guidelines, a number experimental tests were conducted and analysed. The experiments were meant to emulate part of a game engine by simulating movement and collision of a number of entities and measure the difference in CPU time and cache miss rate when the implementation changes, as well as when the number of entities changes. The experiments resulted in four comparisons, which each studied the effect of an isolated change, as shown in Table 1 below.

Table 1: Summary of the comparisons produced by the experiments.

Name	**Intent**
Comparison 1	Observe the effect of removal of run-time polymorphism
Comparison 2	Observe the effect of going from non-contiguous to contiguous allocation
Comparison 3	Observe the effect of reducing the working set size
Comparison 4	Compare an OOD approach implementing the three optimizations above to an equivalent ECS approach

The tests were written in C++ for single threaded execution and compiled with the MSVC 64-bit compiler. The code was then executed on a machine with an Intel Core i7 4770k CPU running Windows 10 x64 and profiled using the Intel VTune profiler.

1.3 Delimitations

Since the concept of Data-Oriented Design is rather wide, the scope is narrowed to only investigating cache related principles, namely limiting the use of run-time polymorphism, iterating over contiguous data, and reducing the size of data used in hot loops.

The performed experiments were limited to measuring the effects on CPU time and the rate of CPU cache misses for single threaded execution of a test-program. Other aspects such as memory usage, or the effect on other caches such as the *TLB* and μop-cache

were not investigated. The execution was also limited to the combination of C++, MSVC 64-bit compiler, and Windows 10 x64 on a single machine configuration.

The thesis focuses on the performance aspect of making code changes. Factors such as maintainability, code complexity or total memory usage are not investigated.

Additionally, only the continuously running part of the program was measured (the game loop). This ignores aspects like allocation and deallocation, which was a conscious choice as these would only constitute a relatively small part of the total workload of a running game.

Regarding the implementation of the test-program, it is also somewhat limited in its scope. It emulates a simplified physics simulation, but leaves out many of the other parts which a game engine might be composed of such as graphics, user interaction, AI, dynamic creation and destruction of entities, to name a few.

2 Background

Computer games can be very demanding applications. They often involve a myriad of dynamic objects whose state must be calculated repeatedly. Whether it be projectiles, particles or AI controlled entities [1]. An important aspect is giving the player a smooth experience. This involves rendering an image between 30 - 60 times every second [1], and sometimes even more frequently. This produces a workload that is fairly unique with its soft deadline of 33 ms (for a frame rate of 30 frames per second) for doing the relevant calculations, while also running for a significant amount of time.

The heavy burden this imposes upon the hardware has led a number of performance enhancing guidelines, which are not necessarily new when looking at all branches of software, but haven't been used very much in the field of game development in the past [2]. This report focuses on some of the guidelines laid forward by proposers of the so-called Data-Oriented Design which are meant to increase CPU throughput by aiding the operation of the CPU cache.

In Section 2.1, the basics behind memory latency are explained. Both about how it affects performance, as well as how the CPU cache may help in this matter. Then, Section 2.2 goes into how the cache is affected by the way code is written, and in what way this is relevant in the field of game development.

2.1 Hardware aspects

This chapter presents the background of why the CPU cache is important for reaching the full potential of the CPU, as well as some basics of its inner workings. The information presented is important for both the reasoning of Section 2.2 as well as for interpretation of the results gathered from the experiments.

2.1.1 The CPU

The heart of any computer is the Central Processing Unit, or CPU. This chip handles most computations that are not graphics related. Any computation that is to be made on a CPU must first be broken down into its simplest form, called machine instructions. An instruction might for example be `ADD`, which tells the CPU to add two numbers. `ADD` is the instruction in human readable form, but in the end, it is fed into the CPU as corresponding binary data - ones and zeroes - often called object code. The set of possible instructions, as well as how they are expressed and interpreted, is determined by the Instruction Set Architecture (ISA) of a given CPU. For desktop computers, the by far most common one is the `x86`-architecture [3].

The CPU does its work in between clock cycles. A simple instruction like `ADD` might be executed in one clock cycle, while a more complex one like `FSIN` (calculating the sine of a floating-point variable) might take many more cycles, also with some variability depending on the architecture of the CPU [4]. But operating at a clock frequency of say 3 GHz, the CPU might execute 3 billion simple instructions per second. And thanks to features like pipelining, a single CPU can process several instructions in parallel which increases this potential rate. Furthermore, the CPUs of today often have multiple cores, each being a separate processing unit, which again multiples the number of potential instructions executed. The bottom line is that the CPU has the capacity to perform a

staggering number of computations each second. But it also needs some data to do its work on, which is kept in the main memory. And herein lies a potential bottleneck, as is discussed in the following sections.

2.1.2 Memory latency

The CPU is only capable of performing computations on values kept in what is called registers, which are storage units located on the actual CPU chip [5]. Each register is usually limited to holding a value of between 8 to 80 bits [5]. Because of the small size, which may account to a few bytes in total for all the registers, only a small portion of the data is ever in that region at any given time. The main memory, on the other hand, may store several gigabytes of data. Transfer between main memory (RAM) and the registers therefore becomes a very important aspect of the normal operation of a computer. Unfortunately, in the silicon-world, there is often a trade-off between speed and size. Fetching data that is in a register may be performed in only a single cycle, but access to the main memory is many orders of magnitude slower [6]. The delay between requesting a value and receiving it is known as the memory latency.

In the early days of computing, memory latency was not a problem. When the CPU needed to access the data at a certain address in the main memory, it could do so in roughly the same time as accessing a value in its registers [6]. But as the hardware evolved and clock speeds increased, the memory latency did not improve at the same rate. This is called the CPU-memory gap (Figure 1) [5]. Nowadays, fetching data from the main memory may take several hundred cycles [6], [7]. Naturally, it would be wasteful to force the CPU to sit idle, or stall, for hundreds of cycles each time a new address must be accessed. Therefore, computer hardware has come to adapt methods to prevent these stalls, and to hide the effect of memory latency when it inevitable appears. But even with these measures, memory accesses may account for upwards of 75 % of total software execution stalls [8].

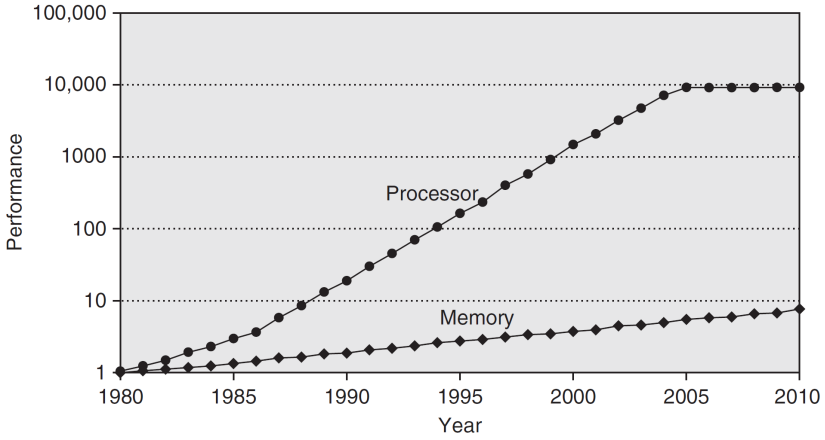


Figure 1: The gap in performance between memory and the CPU as illustrated by the time difference between an on-chip memory request and the latency of a DRAM access. Source: [5]

Nevertheless, for masking the delay caused, the processor may use techniques such as

Vector Extensions, Out-Of-Order Execution and Simultaneous Multihreading [5], [6]. And while these concepts may be mentioned in passing, the main focus lies on the technique of caching, which is an effective method for preventing memory related stalls.

2.1.3 The CPU cache

To mitigate the issue of latency, modern processors utilize a set of intermediate memories between the CPU and RAM called caches. The caches hold no unique data. Instead, they keep copies of selected values from the main memory which are likely to be used in the near future. If data in the cache is modified, it will eventually be copied back to the memory so that all levels of memory are synchronized.

The cache's physical properties, such as being placed closer to the processor, and being of the higher transistor count SRAM-type (unfortunately raising the physical size, power usage and cost) makes the cache a lot faster than the main memory which is of DRAM type [6]. But once again, storage size has must weighed against speed. While the cache is faster than the main memory, it is also smaller. Often only a few megabytes in total on a desktop computer. To battle the size vs speed penalty, a hierarchy of caches is often utilized. Closest to the processor, and smallest in size is the L1 (Level 1) cache, which is remarkably fast. Fetching a value from here may only cost a few CPU cycles [6], but in return, it is only a few kilobytes in size. Then there may be an L2, which is larger and slower, and sometimes an L3 cache which follows the same pattern. In the case of multi-core processors, the higher levels are sometimes shared. A real-world example is the processor used for the experiments in this paper - the Intel Core i7 4770k. Is has an L1 of 64 KiB per core with a latency of 4 cycles, one 256 KiB L2 per core with an 11-cycle latency and an 8 MiB L3 which is shared among all cores and has a latency of 35 cycles [9].

Furthermore, a separation is often made between code and data in the caches. This has been shown to be advantageous as the memory regions needed for code and data are relatively independent [6]. This is also the case for the i7 4770k, which divides its lowest level into 32 KiB reserved for data, and 32 KiB for code. They are abbreviated as L1D and L1I respectively. In the upper levels, code and data are stored together [9]. This is often called a unified cache, contrary to a split cache [5].

2.1.4 Cache operation

The caches are very much invisible to the both the programmer and the compiler during normal operation. Load and store instructions are still written with register- or memory addresses as targets [6]. The CPU along with the Memory Management Unit instead controls the data flow to and from the cache. The end result is that when the CPU issues a read request, it first checks if a copy of the value exists in the closest cache (L1), and, if so, it is fetched from there at a low latency. If not, the next level is checked, and so on. In the worst case, the value is retrieved from RAM. When the requested data exists in a certain level, it is called a cache hit, and when it does not, it is called a cache miss. An access might for example be an L1 miss, but an L2 hit. A miss causes the value to be copied to the level in which the miss occurred, as well as all lower levels.

It would be ineffective to copy a value to the cache only after a miss has occurred since such an event can be rather costly. Therefore, data is always copied in chunks of

contiguous addresses called cache lines [6]. The size of a cache line is decided by the architecture, but it is usually 64 bytes due to it being a good trade-off between speed and miss rate [5], [6]. Figure 2 illustrates the process of requesting one value in memory and causing the surrounding values to also be copied to the cache. Loading multiple values at a time is a central design choice for the operation of the cache, which builds upon which the premise that data which belongs together, for example in the same program, will be located relatively close in memory [6]. This is known as the principle of spatial locality [6].

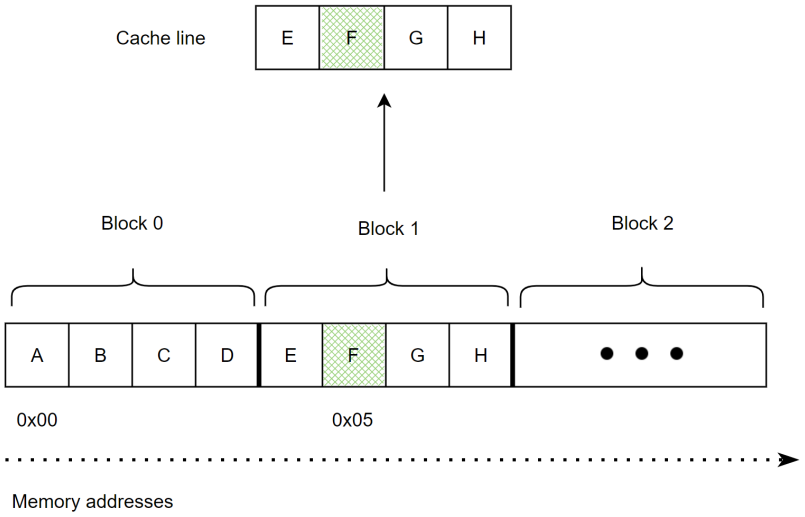


Figure 2: The content of memory address 0x05 (with value F) is requested, and thus all the values residing in the addresses mapped to Line 1 are copied to the cache. The line size is 4 B.

The concept of temporal locality is also a foundation of the workings of the cache. Since each cache miss will cause an entire line to be fetched, another one will have to be replaced, or evicted in other words. Temporal locality is based on the premise that the data recently used has a good chance of being reused in the near future [6]. This is the case when, for example, a loop is iterating over a set of variables multiple times. Therefore, such usage patterns will keep the evictions to a minimum as the number of misses will be kept low.

Writing to a value residing in the cache is a bit more complex than reading a value. The most basic policy for writing is the write-though policy, which means that if a single value in a cache line is written to, it is immediately written to RAM as well [6]. With the write-back policy, on the other hand, the cache line is marked as dirty when a value residing in it is written to [6]. When a dirty line is finally evicted from the cache, the new values are written to RAM [6]. This is a faster, and more commonly used policy, despite being more complex to implement [6]. When memory is shared between cores, however, performance may suffer due to the need of keeping written data synchronized between each separate cache [6].

2.1.5 Associativity and its relation to performance

Since the ratio is not one-to-one between the number of lines fitting in a certain level of cache and the number of lines fitting in the RAM, there must be a mapping between the two memory locations. There are different ways to solve this. The simplest way is a direct mapped cache, which means that a block in memory will always go to the same slot in the cache. Block n in memory would thus be mapped to line

$$\frac{total\ size}{line\ size} \bmod n$$

in the cache, as shown in Figure 8. If the cache is of size 32 KiB, for example, it means that it is able fit 512 cache lines of 64 B each. If line n resides in the cache, and block $n + 512 * k$ is fetched, the line would have to be evicted to make room for the newly fetched one. When a cache miss causes the eviction of another line, even though the total capacity is not the issue, it is called a conflict miss.

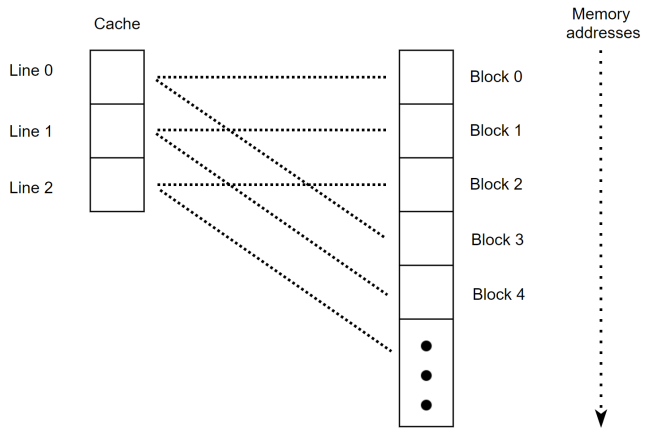


Figure 3: Mapping between main memory and a direct mapped cache with a capacity of three lines.

Another way of mapping, which reduces this type of misses, is set-associative mapping [5]. This mapping divides the cache into sets, each of which holding a number of cache lines. An m-way set associative cache means that each set fits m different lines. Figure 4 illustrates this concept. The number of sets in the cache would be calculated by:

$$number\ of\ sets = \frac{total\ size}{line\ size \times associativity}$$

If we imagine a continuous iteration over memory which brings new lines into the cache, the first line in memory would be mapped into set 0, the second one to set 1, and so on. When line k * $number\ of\ sets$, is reached (k being any integer multiplier), it will again be mapped to set 0. But now the CPU has a choice of which of the m lines residing in Set 0 to replace. The optimal choice would be the one that is the least needed. But as this is hard to know for sure, an educated estimation must be made. The algorithm

used for the choice is called the replacement policy. Usually, processors utilize a Least Recently Used policy, which replaces the value that has not been read or written for the most amount of cycles [6]. By using a low-cost heuristic, a fast approximation of this value may be made, sometimes giving it the name *Pseudo-LRU* [5].

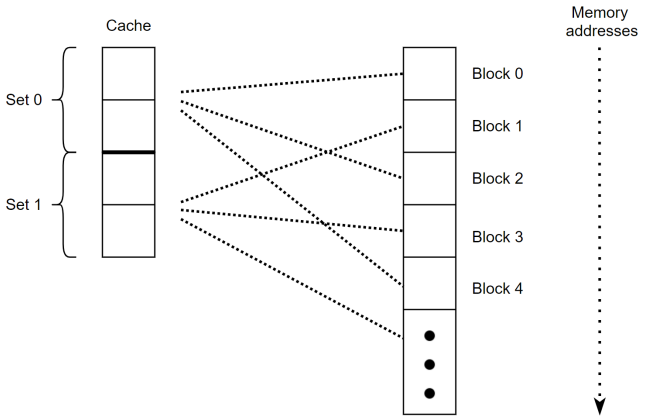


Figure 4: Mapping between memory and a 2-way set associative cache with a capacity of four lines.

At the very extreme, when all lines are mapped to a single set, it is known as a fully associative cache. This is seldom practical, however. As seen in Figure 5, the cache misses decrease with an increased associativity, but with diminishing returns. Figure 6 shows the drawbacks in the form of increased latency when associativity is increased. This is an effect of requiring a higher number of comparators (a type of electrical circuit) causing higher look-up times as well as an increased cost and space requirement [6]. For this reason, 8-way associativity is seldom exceeded for non-shared caches, while shared caches might benefit from an even higher associativity [6].

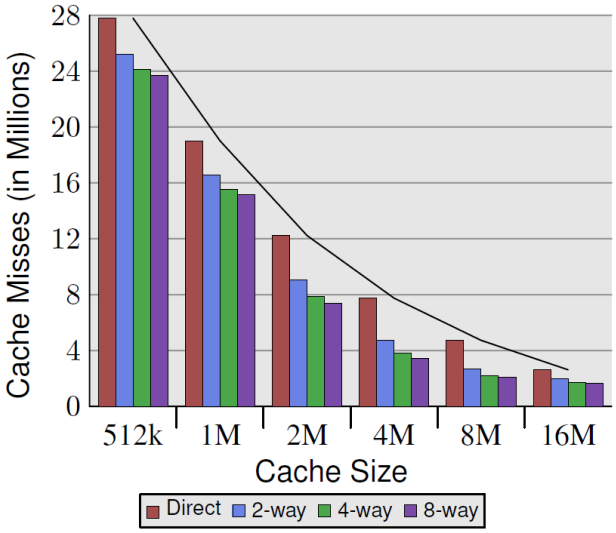


Figure 5: Relation between associativity and the number of cache misses.
Source: [6]

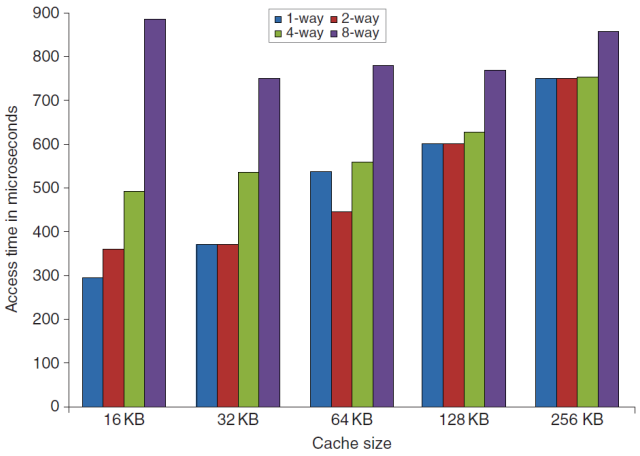


Figure 6: Relation between access time and associativity.
Source: [5]

2.1.6 Prefetching

To reduce the number of compulsory misses, which is the type of miss caused by the first access of a new variable, data may be prefetched into the cache. This means that a cache line is brought in before it is needed. A prefetch may be triggered by the software, in the form of a special instruction, or, more commonly, by the hardware prefetcher [6]. The prefetcher automatically identifies usage patterns and brings in the appropriate lines. For example, assuming a line size of 64, if a loop iterates in steps of 1 to 64 bytes at a time, the prefetcher would keep the next following block ready in the cache. But if the strides are larger, it would make more sense to skip certain blocks and for example bring in every other or every third one. And if the prefetcher identifies a loop going backwards, it would instead bring in a previous adjacent block [6]. There often exists some threshold though, such as a minimum of two sequent misses to start prefetching [6]. Otherwise, random access, or sporadic access to a global variable would trigger unnecessary prefetches [6]. Also, prefetching is limited to cache lines residing in the same memory page as the one(s) which triggered the prefetch [6]. If it was not, a prefetch could trigger a page fault [6], which in the worst case could cause page-walk costing thousands of CPU-cycles. The prefetcher may keep track of multiple simultaneous patterns of access, each pattern being called a stream [6].

2.2 Programming

To make the most of any given hardware, the programmer must be aware of not only of the efficiency of the high-level algorithms used, but also the finer details of the implementation. An example is iteration over a 2-D matrix. The choice between a row-major order or a column-major order of iteration should not matter, on paper. But due to the way it affects memory accesses, the row-major order is faster to perform in most languages, as it is more common to store the arrays in a row-wise order in memory, resulting in faster accesses [6], [10]. The compiler may often invisibly assist by identifying and applying optimizations, but still, there is no guarantee that the compiler will always see the programmer's intent, so one must sometimes organize the code in such a way that it is not completely bewildered.

The bottom line is that it is beneficial to know about the details of both the hardware limits as well the implementation details of the programming language and compiler. Since the cache is the focus of the report, it is also the center of the following sections, which explains how different programming styles may affect the data and instruction caches. It focuses on aspects related to game programming, and often to C++ in particular, since this was the language used to implement the tests in this paper, as well as it being commonly used in game development [11].

It is to be noted that the behaviour of the cache can differ when multi-threaded execution on multiple cores is brought into the picture. This is due to the necessity of keeping shared memory synchronized between cores, introducing the need to maintain cache coherency [6]. It also introduces the possibility of false sharing, even when variables are not shared between threads [6]. These aspects are disregarded, however, as the thesis limits the scope to examining single threaded execution.

2.2.1 Data locality

As is discussed in Section 2.1, accessing contiguous data is fast. The first access to a variable might incur a cache miss, but after that, access to the remainder of data in the fetched cache line will be near instantaneous. The hardware prefetcher will also be able to keep the caches loaded with a higher fraction usable data if adjoining memory addresses are accessed. Furthermore, if values that are used together are also stored closely together in memory, it reduces the probability of them being mapped into the same set of a set-associative cache, thus reducing the number of conflict misses [6].

The programmer has a multitude of choices when choosing the structure for storing data, but some choices honor locality more than others. A pointer-based structure such as a linked list may in theory sound fairly good with its $\mathcal{O}(n)$ time complexity of linear traversal, but will in most implementations have a discontiguous memory layout. An array will have the same time complexity for a linear traversal, but due to - for most implementations - storing its elements contiguously, the real-world performance will likely be better [8]. Figure 7 and 8 shows the difference in memory layout between the two structures.

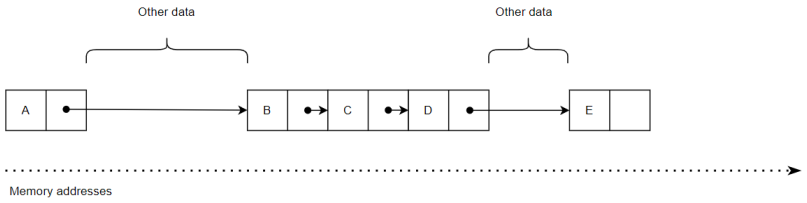


Figure 7: Possible discontiguous allocation of five elements in a singly linked list.



Figure 8: Allocation of five elements in an array.

There are some differences between languages. An array, or a `vector` from the C++ standard library, are specified to guarantee that the elements are contiguously allocated while Java, for example, gives no such guarantee for its own Array data structure [12]. Sometimes there also exists a difference between data types. In C++, both the `struct` and `class` can be specified as either value of reference types, but other languages, such as C#, differentiate between the two. In the case of C#, `struct` is a value type, while `class` is a reference type [13], meaning that it is accessed by a pointer and automatically allocated on the heap. This would make it difficult to achieve a contiguous storage of objects in such a language.

2.2.2 Structure size and alignment

Because of the way memory is loaded into the cache - in lines of multiple bytes at a time - if only a one or a few members of a data structure are used for a computation, the whole corresponding line will still be copied to the cache. This can lead to situations where most of the data inside the line remain untouched, which lowers both the fraction of usable data in a single line, and also translates to the utilization of all cache levels, leading to capacity misses which could have been avoided.

Using a `short int` of 2 bytes size in favor of a regular 4 byte `int` might seem petty with today's abundance of storage, but might for the reason above shave off enough of the working set as to keep the most needed data inside the cache, or at least waste less of its size.

It is also worth noting that care must be taken when mixing variables of different size inside a class or struct. To get the lowest memory usage, and therefore aid the caches, member variables should be ordered by their size. The underlying reason is that the CPU accesses memory faster if data is *naturally aligned*, meaning that variables are located on memory addresses which are multiples of their size [14]. This is at least the case for `x86`, while other architectures might even require alignment to work properly [1], [5]. Inside a data structure, this has the implication that memory access is faster when data is aligned to the size of largest member. The compiler will therefore, when necessary, add padding to keep also the smaller variables aligned to the same boundaries [14]. The effect is seen in Figure 9 below, where interleaving variables of different size causes the compiler to add padding, thus increasing size without adding any useful information. In Figure 10, the size is decreased by 4 bytes simply by moving the 4-byte integer to the beginning of the structure.

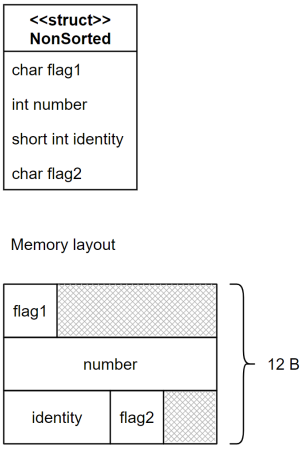


Figure 9: Structure with unsorted variables causing the compiler to add padding.

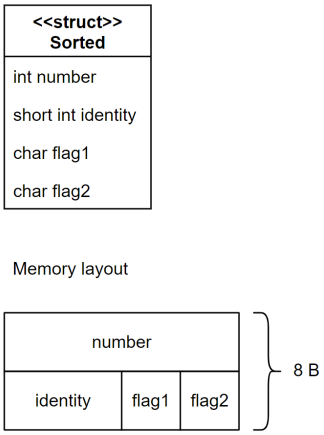


Figure 10: Structure with variables sorted by size helps keeping data packed in memory.

2.2.3 Instruction locality and size

The same principles of locality and size as discussed earlier also applies to code. Generally, though, the programmer has a more limited control on affecting the locality, and thus cache efficiency, when designing code compared to when making choices about data [6]. This is due to a number of reasons. Firstly, the size of the code is often proportional to the problem to be solved, and therefore relatively fixed [6]. The compiler also has a lot of room for optimizing the generated machine instructions when it detects the programmer's intent [6]. And lastly, the CPU is better able to detect access patterns in code than in data patterns [6].

But there do exist ways to take advantage these principles. It is still of importance to keep code compact [6]. This sometimes goes against techniques used by the compiler known as inline expansion, or *inlining*, and *loop unrolling*. Inlining means that instead of translating a function call to the normal `call` and `return` instructions in assembly code, the compiler may instead copy the entire function and its arguments to its call location, thus eliminating some of the steps involved in a function call in an effort to

lessen the overhead. This often produces larger code, especially when the function is called more than once, and may in that way put a larger strain on the cache [6]. Loop unrolling builds upon the same principle of lessening the overhead of loops by lowering the number of steps involved for each iteration by generating a larger amount duplicated code. Both techniques may be limited by using a compiler flag such as `-Os`, or by adding the `-noinline` attribute to the function in question [6].

Another way which may lessen the cache load, is by separating seldom used code by introducing a new function. If the programmer knows that a certain part of a frequently accessed code block has a low probability of being reached, for example code in a conditional statement, that code might be better suited as a separate function. This increases the overhead by introducing another function call, but if the displaced code was large enough, the cost may be offset by the lower size of the frequently executed code region [15].

2.2.4 Run-time polymorphism

Polymorphism is a feature used in Object-Oriented programming languages, which allows for objects of different types to use the same interface [1]. An example, inspired from Jason Gregory [1], would be classes of the type `Rectangle` and `Circle`, which both are in need the function `Draw()`. If they both inherit from the base class `Shape`, this class would specify the function interfaces, while the specific implementation lies in the derived classes of `Rectangle` and `Circle`.

There is a difference between run-time and compile time polymorphism though. If the implementation to be used is known when compiling, implying that the derived type is known, the compiler can solve the function call in the same way as a non-polymorphic call. This is called compile-time, or static, polymorphism [16]. If, on the other hand, the function version is not known at compile time, such as when a function is called on an object of type `Shape`, it is instead called run-time polymorphism, or dynamic dispatch [16]. One of the benefits of this is that it enables the program to iterate over a collection of Shapes and call the `Draw`-function, without knowing at compile time if a specific element a `Rectangle` or a `Circle` [1]. There are some drawbacks to dynamic dispatch, however, as it complicates the actual machine code implementation, which adds an overhead of a couple of extra operations per function call [17]. It may also hinder the compiler from performing inline expansion since the function address is not known at compile-time [18].

The actual implementation may differ between languages and compilers, but in many cases, it is solved by using a function lookup table called a virtual table [19], as is the case for C++ [20]. This means that each polymorphic class has a pointer to a table, which in turn contains pointers to the different implementations of the function with a shared interface. The table is often referred to only as the `vtable`, and the pointer to it as the `vpointer` [21]. The `vpointer` is invisible to the user since it is added at compile time, but will usually be the first or last member of each object.

Focusing on cache usage, the extra layer of indirection due to the `vtable` may affect the instruction cache in a negative way [6], [22]. Also, every polymorphic object will contain a `vpointer` of 4 bytes on a 32-bit system or 8 bytes on a 64-bit system, or any multiple of these when implementing multiple inheritance [1]. The effect is that it lowers the data

cache utilization since the pointer will take up a relatively big portion of a cache line of say 64 bytes, while still not being of practical use in any computation. The matter is made worse if the `vpointer` resides in a cache line separate from the member variables needed for the function. In that case each virtual function call will first cause the line of the `vpointer`, and then another containing the actual data needed, causing a very low utilization at the cache line level [2].

Another effect of implementing polymorphic objects in C++ is that they cannot be directly stored in a safe way in the standard data structures of C++ e.g. an array or `std::vector`, since they do not have a fixed size [23]. When instead using pointers, there is no guarantee that the actual values they are pointing to will be located contiguously in memory. Likely, they will be allocated relatively close, but if the system is dynamic and objects have to be created and destroyed during runtime, as is often the case in a computer game, the memory will become more and more fragmented, leading to more cache misses when iterating over the objects. There are, however, methods to resolve this, such as custom allocators [24].

When keeping a collection of polymorphic objects, it is of importance for the instruction cache utilization to keep them sorted by type. If each iteration step results in a different type of object for which a virtual function is called, then if the function sufficiently large, or if a large enough number of different functions are called, the least recently used ones will be evicted from the cache. But if it is possible to first process objects of type A, then B, then C, etc., evictions from the instruction cache are less likely [2], [18].

There are also other costs of implementing run-time polymorphism not directly linked to the cache. Iterating through a list of polymorphic objects, each time the version of the called function is changed, a branch misprediction might occur, costing upwards of 20 CPU-cycles. This again shows the use of keeping lists of polymorphic objects sorted by type [21].

2.2.5 Game development and OOD

The Object-Oriented Design (OOD) has been around for many decades [25] and is often the pattern of choice for programming courses. The reason for this is perhaps thanks to its easy analogies to the real world, which comes in the form of modeling objects in the code after their real-life counterpart [26], [27]. While there is no official definition of the design [25], it often promotes certain approaches to solving problems, such as inheritance, polymorphism and keeping each object self-contained with its own code and data with an outwards interface, also known as encapsulation [25]–[28]. An objective is often to make the code readable and modular [26], [27].

OOD is deep seated among game developers [2], [26], [29], and many game engines are built upon the design principles. This includes Godot [30], OGRE [31], as well as Unreal Engine [32], to mention a few. Large inheritance trees of polymorphic objects, as well as iteration over collections of said objects is a common sight in game engines [2], [33], [34]. These practices might affect data locality as well as increase the object size as described in sections 2.2.1 to 2.2.4. As an example, Unreal Engine has the base class *Actor* which all objects that can be spawned in a level inherits from. It contains over 100 member variables of different size [35]. Now, the implementation is not open for inspection, so there may be some optimizations behind the scenes, but it is still unlikely that the end

user of the engine will not feel some impact of inheriting this data for their classes.

As is apparent, the use of OOD does not enforce these mentioned, less than cache-optimal, principles, but it seems like it may enable them, at least in the field of game development. There is, however, an emerging trend towards adapting more cache friendly approach, described in the next section.

2.2.6 Data-Oriented Design

Data-Oriented Design is an approach that has been around for some time, but the actual term was perhaps coined, or at least popularised, when Noel Llopis published an article about the concept in 2009 [2], [29]. It can be seen as a series of principles, in which the data flow is the fundamental part of the design [28]. This goal is to enable a more effective use of the cache, as well as aiding vectorization and parallelization [2], [34], [36].

By being a loose set of terms and not a fixed set of rules, it not necessarily disjoint from Object-Oriented Design. Performance centered OOD actually utilizes many of the methods proposed by DOD [37]. But, compared to what is commonly seen in game development, much of it goes against the usual practices of OOD.

While OOD keeps state and behaviour closely tied together, in DOD they are often separated. We are thus left with a set of functions operating on a set of plain data. The data should also be sorted so that it exists in the form of homogeneous chunks [29], [34], [38], such as an array of only position-integers, or an array of only area-floats. Simple, predicable, and linear data structures should be favored in place of higher-level structures like lists and hash tables [36]. A Structure of Arrays (SoA) structure layout is often used to achieve said objectives [2]. In the end, these guidelines enable batch processing of data, broken down into sequential steps of separated logic, which promotes an effective use of the cache by keeping spatial and temporal locality high of both data and instructions.

Run-time polymorphism is also avoided as best as possible [2], [34], [38]. Instead, other workarounds are preferred, such as keeping separate arrays for each type [38] or by splitting the common functionality into a separate data type [34].

A focal point of DOD is keeping data packed in memory. This can be done by avoiding sparse arrays, especially in hot sections of the code. Instead, it might be worthwhile to copy the data into a new, packed, array if one or more elements are no longer needed in the array being processed. The cycles lost by copying might be compensated for by the more effective use of the data cache when iterating over the packed array [29], [34], [38].

Another technique to keep data packed is the use of hot/cold splitting [38]. This means that data is separated so that frequently used data is kept in one container, which may be looped over in a continuous manner. The less frequently used data, such as a victory animation that is known to only be used once in the lifetime of the program, or some flag that is checked once per second, and not once every frame, is kept together in another container which may be connected to the hot data through a pointer, if needed. The cost of the compulsory misses that occur the few times this data is needed is offset by

the better cache utilization when processing the commonly used data.

It is not always the case that the whole game engine needs to be written in the style of DOD. Often, one identifies the common cases, such as operations that will happen thousands of times every frame and focuses the effort on optimizing them, instead of the operations that might only happen once every frame [29], [36]. Systems like the GUI might still be fine written in a more traditional style, when looking at the overall performance of the program [29].

Some game engines have adopted DOD in parts of their implementation or as a whole. Unreal Engine moved to a Data-Oriented Design pattern for their cinematics subsystem in 2020 [39], and Unity Technologies introduced their so called Data Oriented Technology Stack for the Unity Engine in 2019, along with the Entity-Component-System framework[40], which is described in the next section.

2.2.7 The Entity-Component-System pattern

The following section briefly describes the Entity-Component-System (ECS) pattern to give a more tangible example of how the DOD principles may be applied in a game setting. As mentioned, the pattern is based upon the principles of DOD, and is used in the Unity Engine, as well as in many other engines and frameworks. Bear in mind that there does not exist a unified definition of the pattern. Therefore, a very generic and reduced example is given, loosely based on descriptions in from Cristiano Ferreira and Mike Geig from Intel [41], and Niko Savas, creator of the Nomad Game Engine [42].

The architectural pattern features a component-based design, which uses composition rather than inheritance to give game entities different behaviour. As one might derive from the name, Entities are the equivalent of Objects in the game world. But they are not implemented as objects or structs, Instead, they simply exist as an integer which is used to separate different entities for example when looking up a value in an array. Not unlike a key in a relational database [2]. As an example, the entity with identity 0 might be a Human with a `Position`, a `Velocity`, some data for rendering and some number of `Health`. To supply this entity, being only an integer, with this functionality, Components and Systems are used in conjunction.

The components store only plain data. In this example, a `Velocity`-component would be needed, which could be a float. Also, the entity would have a `Position`, which could be in the form of the separate components `XPosition` and `YPosition`, each being a float. Alternatively, a `struct` could keep the two positions together, depending on the implementation. Most likely, the latter would be chosen, as both the `x` and `y` components would be needed for most functions operating on a position. Finally, it could have a `HealthComponent`, for which a char would suffice since the health only needs to have a value of between 0 and 100. All in the spirit of keeping the data as small as possible.

Then, there are the so-called Systems, which contain the code needed for the game logic. Each System operates on only the select entities which contain all components necessary for the System's operation. To continue the previous example, a `MovementSystem` might iterate over all entities with both a `Position` and a `Velocity` and do some computation which alters all the positions in the world. A simpler `HealthSystem` would operate on

all `HealthComponents`, and perhaps decrease its value with one tick for every entity each time it is run. Both systems would be able to linearly iterate upon one or two arrays of consisting entirely of relevant data for the logic performed, resulting in favorable conditions for good cache utilization. In Figure 11 below, a schematic of the described architecture is shown, and in Algorithm 1, the corresponding pseudo game loop.

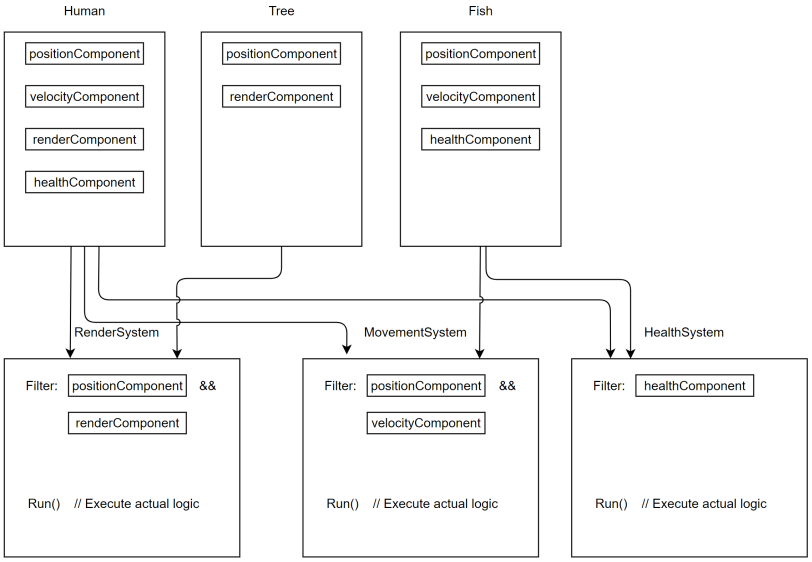


Figure 11: Schematic for an ECS design with three types of Entities and three different Systems.
Adapted from [41]

Algorithm 1: `Pseudo game loop for an ECS based game engine`

```
1 while gameIsRunning do
2     movementSystem.Run();
3     healthSystem.Run();
4     renderSystem.Run();
```

The example is not yet complete, because there needs to be a way to link every entity to its respective components. A simple way would be to initialize an array with a given size for every component. If we know that the game will never have more than 1000 entities, this would be a suitable size. Entity 0 would then have the data of `positions[0]`, `velocities[0]` and `healthComponents[0]`. This would of course mean that if not all entities in the game have, for example, a `healthComponent`, those components would be stored in a sparse array. An example of such of a storage layout is shown in Figure 12. There are ways of avoiding sparse arrays, thus keeping data packed for better cache utilization, by for example having a Manager-class keep track of the mapping between the entity-ids and their components [42].

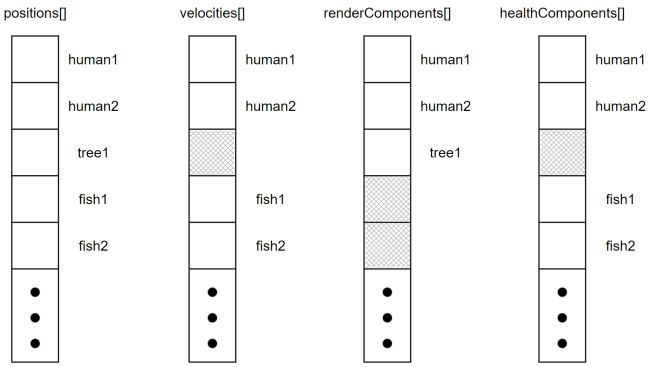


Figure 12: ECS memory layout for four types of components mapped to five different entities.

Also, when the system is dynamic, as in that entities are destroyed and created during run-time, which is often the case in games, such an event would need to be addressed. One solution could be to reorder the Component-arrays when an object is destroyed and keep the non-active identifiers in the end of the array. In a larger scale game, some type of manager would probably be used for this too, but the basic underlying principle would be the same.

3 Related Work

Similar studies have been done, looking at the effect on CPU performance when implementing a Data-Oriented Design. In 2018, Walid Faryabi [43] looked at the frame rate and CPU time for a series of tests which compared an OOD based approach to a DOD approach. The first test was a sine wave rendering built using the ECS pattern, using OpenGL for rendering. The second one was the same sine wave test, but built for the Unity Engine using Unity's own ECS framework and rendering engine. The third test was a partial conversion of a commercial game called Dwarfheim built in the Pinecone Engine (which is built on top of the Unity Engine), from being based on a OOD design, to being partly built using the ECS framework of the Unity Engine. All tests were written in C#.

The results were mostly in favor of the ECS design, with a decrease in CPU time as the entity count increased. Some tests showed upwards of a 1500 % performance increase. Profiling in Unity showed that the second test was sometimes GPU-bound, and Faryabi suspected that it would have shown a larger performance difference if was not. The Dwarfheim conversion did not exhibit any performance gain after the ECS conversion when tested on large number of entities with rendering turned on, but did perform better with in every case with rendering turned off. This was attributed to there being a lot of dependencies of code still built upon a OOD design. Faryabi speculated that the linear data layout and thus better cache utilization likely stood for the performance increase, but he concluded that it was not clear if performance gain could entirely be attributed to the Data-Oriented principles [43].

In 2019, Dennis Nilsson and Anton Björkman [44] compared the performance, in terms of rendering time, for a game engine based on either a Data-Oriented Design or an Object-Oriented Design. The engine featured moving circles, bouncing on the screen bounds, and being deactivated if two circles collided. The circles were rendered with the SFML-library. The engines were written in C++ and implemented using an ECS framework called EntityX, which also featured a benchmarking tool which was used to measure performance.

They came to the conclusion that the Data-Oriented design performed better exceeding 3500 simultaneous entities, and theorized that the performance gain was due to better cache utilization since the data size of 3500 entities in the OOD implementation corresponded to the total size of the four L1D caches (one for each core). The details of the implementation were sparse though, so one must assume that multiple threads were used in the test program for their theory to be sound. Their conclusion was that it is worth considering a Data-Oriented Design when developing games that are reliant on performance, and suggested profiling the number of cache misses as a recommendation for future research [44].

Furthermore, in a paper by Gustav Löfstedt [45], an aggregation-based entity system (a game engine, in essence) was compared against a Data-Oriented entity system with *entities updated per second* as the metric. Additionally, the Last Level cache misses (L3) were profiled with the VTune profiler on an i7 720QM with 6MB of L3-cache and presented as the metric *cache misses per second*. He observed a consistently higher number of entities processed per second for the Data-Oriented system when testing on different numbers of entities. Also, the number of entities removed and created during

runtime was varied. He saw an increase in the number of cache misses for the DOD engine when the dynamics was increased, but with no large visible differences for the OOD engine. The effect was hypothesized to either originate from either an increase in memory fragmentation or from an increased number of memory accesses due to the inner workings of the `stl::multimap` data structure which was used to store the components of an entity.

Ultimately, he associated some of the performance increase of the DOD engine to the lower number of cache misses. He also points out that some of the tests showed an increase in performance with no change in cache misses, and speculates that there might be differences between the implementations that do not only affect the CPU though changes in cache utilization [45].

4 Method

Based on the background given in Section 2.2, a series of experiments were devised to test the effect of four different changes in coding practices related to cache utilization. The experimental approach was to implement a game loop performing the same task in five different ways and running it on a varying number of entities. The variation of the entity count was meant to illuminate any differences related to the size of the working set (essentially, the amount of data used in the loop).

As a metric for performance, the iteration count when running the loop for a fixed time was used, which is inversely related to the CPU time needed. Also, the cache miss rate for the different caches were used as metrics for evaluating each implementation.

Since the area of interest is mainly game development, the experiments were all centered around an algorithm mimicking what might be part of a game engine, namely movement and collision detection in two dimensions. It is similar to the test used by [44], but does not feature entity-entity collisions or rendering.

The changes between the five implementations were related to patterns often seen in game development and the corresponding corrections suggested by the Data-Oriented Design principles as described in Section 2.2.5 to 2.2.7.

The first implementation uses polymorphic classes referenced through an array of pointers, calling two virtual functions for each entity every iteration of the loop.

Implementation 2 is a modified version of the first implementation which removes the use of virtual function calls.

Implementation 3 does away with the use of polymorphic classes. Instead it used regular classes stored in a contiguous array. This is compared to Implementation 2 to observe the effect of going from non-contiguous to contiguous allocation.

Implementation 4 is the same as Implementation 3, but reduces the size of the entities by removing unnecessary variables. Comparing the two shows the effect of keeping the down structure sizes. Implementation 4 also represents a cache friendly Object-Oriented Design, as it does not feature run-time polymorphism, iterates over a contiguous array, and does not use larger structures than necessary.

Implementation 5 is a larger reworking of the code in an Entity-Component-System inspired way. The results gathered when running this implementation were compared against the cache friendly Object-Oriented Design of Implementation 4 to examine if it could produce better performance.

The implementations were all limited to single threaded execution. They were written in C++, compiled with the MSVC 14.28 64-bit compiler, and executed on a machine equipped with an Intel Core i7 4770k running Windows 10 x64. Intel VTune was used for profiling cache misses. Each implementation was tested at multiple times to reduce the effect that background processes might have had on the results.

In Table 2 below, the changes between the different implementations are summarised. The base algorithm is described in further detail in Section 4.1, while a more in-depth

description of each change is given in Section 4.2. Finally, Section 4.3 to 4.6 present the specifics of the testing environment and test parameters.

Table 2: Summary of differences between the implementations

Designation	Design	Working set size [bytes per entity]	Virtual function calls	Contiguous allocation
Implementation 1	OOD	56 B	Yes	No
Implementation 2	OOD	56 B	No	No
Implementation 3	OOD	56 B	No	Yes
Implementation 4	OOD	20 B	No	Yes
Implementation 5	ECS	20 B	No	Yes

4.1 Collision detection algorithm

All implementations features the same high-level algorithm: a simulation of movement of 2D-rectangles and collision detection between the rectangles and four enclosing walls. This emulates parts of a game loop, where this type of physics simulation may happen multiple times between each rendered frame [46]. The basic steps are outlined in Algorithm 2 below.

Algorithm 2: `Movement and collision detection (high level)`

```
1  create entities
2  start measurements
3  while time < timeLimit do
4      calculate delta time
5      increment counter
6      for each entity do
7          simulate movement
8      for each entity do
9          calculate boundary collisions
10         correct collisions
11 stop measurements
```

First, a number of rectangles (a term used interchangeably with *entities*) are initialized with a random x and y position in the form of a floating-point number. The x-value may vary between 0 and 1920, and the y-value between 0 and 1080. This is, not coincidentally, a common monitor resolution, and was used for debugging purposes. The rectangles are given a *side length* of 1 in the form of an integer. The reason for not using a float was mainly to enable them to be drawn as a single pixel when debugging. Then, an initial *velocity* is randomized for each pixel in the x and y direction of between -0.05 and 0.05. This value and decides how much each rectangle will move for each iteration of the loop. A *velocity* of 1 would cause a movement of one position value (one pixel) every millisecond.

When the entities have all been initialized, a timer is started, which measures the elapsed loop time using the `steady_clock`-struct from the C++ `std::chrono` library. Also, a function call is made through the VTune API to signal that data may begin to be collected from the CPU's hardware counters if the program is run inside the external

VTune profiling environment. The game loop (Line 3) is then repeated as many times as possible for a fixed time.

At the beginning of each iteration, the current time is fetched. This is used to calculate the time difference in nanoseconds since the last loop, also known as the *delta time*. Then, a counter is incremented, which is used to measure the final performance. A larger count means that more computations could be made in the time given, which produces a metric of how a certain implementation fared in terms of CPU time.

All rectangles are then iterated over in an inner loop. First, they are moved to a new position, determined by multiplying the current position in both axes with the *velocity* and the *delta time*. The time-step factor results in a constant motion regardless of the execution speed of the loop.

When all rectangles have been moved, they are iterated over again, and each rectangle is checked for intersection with the boundaries. The boundaries are set to a rectangular area with an upper left coordinate of (0,0) and lower right coordinate of (1920, 1080). In accordance with the standard of the `SDL`-library, used for debugging, the x-coordinate is positive to the right, and the y-coordinate is positive in downwards direction. If an intersection with a boundary is detected, the rectangle is moved back with the distance of the overlap and half of its *side length*, and its velocity in the normal direction is reversed. This gives the appearance of them bouncing against the four walls. The backwards movement is an emulation of a technique used to prevent tunneling, which is a common phenomenon in game physics simulations where an object may pass through a boundary if the time-step is too long, i.e. the game is running too slow [11]. In this algorithm though, tunneling would not happen with a too long time-step. Instead, the object could get stuck in a loop where the velocity changes back and forth from positive to negative, which this precaution protects against. In Figure 13 below, a boundary collision is illustrated. The loop runs in $\mathcal{O}(n)$ time complexity since each rectangle is ever only compared to two coordinates in each iteration, regardless of the total number of entities.

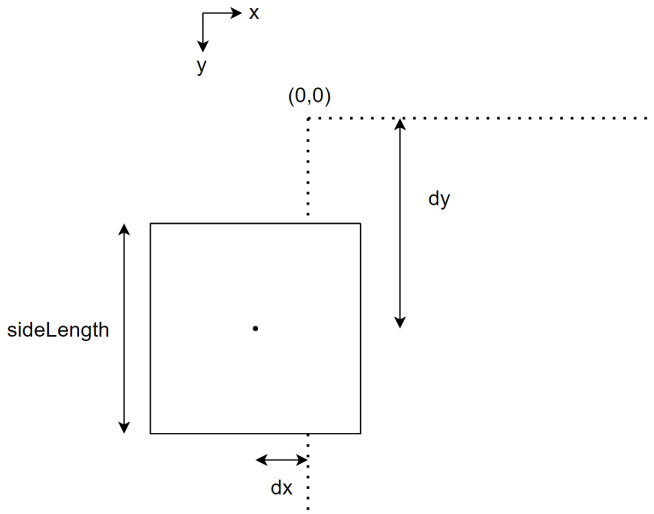


Figure 13: Rectangle colliding with the left boundary. It will be moved to the right by distance $dx + sideLength$.

When the game loop has run for the pre-set time, the timer is stopped, and the loop counter is printed to the console. No clean-up or deallocation is done as the program is restarted between test runs.

4.2 Implementation details

In this section, the finer details of each implementation are presented.

Implementation 1 - Polymorphic classes and virtual function calls

The first implementation features a class hierarchy in which the `Rectangle`-class is derived from the base class `GameObject`, as shown in Figure 14 below. `GameObject` is composed solely of two pure virtual functions, `Move` and `CheckBoundaries`, which the class `Rectangle` overrides. Apart from the member variables necessary for the computations, `Rectangle` is also made up of a `name`-variable, which simulates some less often used data. This makes the total size of the structure amount to 40 bytes.

The reason for the extra data was to keep the working set size the same for Implementations 1 - 3, so that Implementation 3 could be reused to look at the effect of lessening the structure size. This was an unfortunate compromise to lessen the number of different implementations, and one which likely slowed down the execution more than if the burden had only been the use virtual functions. But as the same is true for Implementation 2, the relative performance between the two should not have been affected. One can only expect to see a lower absolute performance than if the class sizes were kept to a minimum.

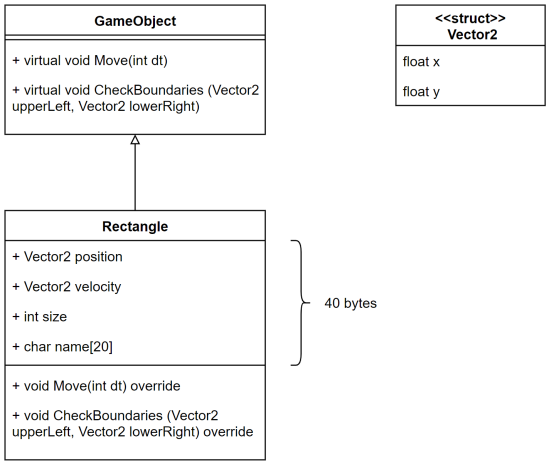


Figure 14: Class diagram for Implementation 1. `vpointer` not shown.

The Rectangles to iterate upon are dynamically allocated and referenced though pointers to the base-class (`GameObject`) stored in an array. The array resides on the heap so as to not reach the default stack limit of 1 MB set by MSVC [47]. The initialization is shown in Listing 1 below. With the array of pointers and the hidden `vpointer` included, the working set when running the main loop adds up to $40 + 8 + 8 = 56\ B$ per entity.

Listing 1 Initialization of Implementation 1

```
GameObject** rectangles = new GameObject * [entityCount];

int size = 1;
for (int i = 0; i < entityCount; i++) {
    /* ... Calculate random starting values ... */

    rectangles[i] = new Rectangle { size, posX, posY, velX, velY };
}
```

The game loop is shown in Listings 2 to 4 below. Each iteration invokes two virtual function calls, both of which having to face the indirection of dereferencing the pointer to the *vtable*, and then the overhead of doing a lookup of the actual function address.

Listing 2 Game Loop used in Implementation 1 to 4

```
while{timer.elapsedTime < targetTime} {
    /* ... Calculate time difference since last iteration ... */

    for (int i = 0; i < entityCount; i++) {
        rectangles[i]->Move(deltaTime);
    }
    for (int i = 0; i < entityCount; i++) {
        rectangles[i]->CheckBoundaries(upperLeftCorner, lowerRightCorner);
    }
}
```

Listing 3 Overridden `Move`-function

```
Rectangle::Move(int deltaTime) {
    posX += velX * deltaTime;
    posY += velY * deltaTime;
}
```

Listing 4 Overridden `CheckBoundaries`-function

```
1  Rectangle::CheckBoundaries(Vector2 upperLeftCorner, Vector2 lowerRightCorner) {
2      float dx;
3      float dy;
4      float sideLengthHalved = (float)(size / 2);
5
6      // Check left boundary
7      dx = position.X - upperLeftCorner.x;
8      if (dx < sideLengthHalved) {
9        velocity.x *= -1; // Reverse velocity in normal direction
10       position.x += (sideLengthHalved - dx); // Move back
11     } else {
12          // Check right boundary
13         dx = lowerRightCorner.x - position.x;
14         if (dx < sideLengthHalved) {
15           velocity.x *= -1;
16           position.x -= (sideLengthHalved  - dx);
17       }
18     }
19
20     /* Check upper and lower boundaries as in the same way as in Line 7 - 18 */
21 }
```

Implementation 2 - Removal of virtual function calls

To test the effect of using regular function calls instead of virtual calls, Implementation 1 was is changed slightly. The difference is that the `Rectangle`-class now implements its own `Move-` and `CheckBoundaries`-functions, as shown in Figure 15. Therefore, the function calls should not be affected by any `vtable`-pointer indirections. To make the test only reflect the change in the way function calls are performed, the other parameters were kept the same. Rectangles are allocated with the `new`-keyword and referenced by pointers as before, but the array now consists of pointers to the derived class instead of to the base class, see Listing 5. Also, the base class still contains a (non-used) virtual function, which only serves to make the compiler produce a *vtable*-pointer, thus keeping the size and layout of the class the same as before. Using `sizeof` operator showed that the size still amounted to 48 bytes. Including the pointer to each entity, the working set now adds up to $48 + 8 = 56$ B per entity.

The test was also intended to give an indication of how iterating over a pointer-based data structure, such as a linked list, containing non polymorphic objects would perform, since this would also entail non-contiguous data in most cases.

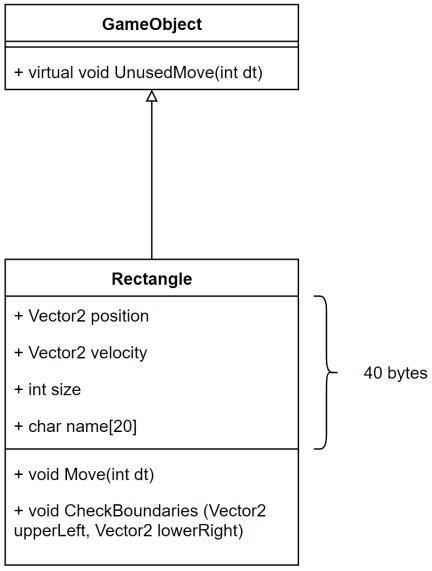


Figure 15: Class diagram for Implementation 2. `vpointer` not shown.

Listing 5 Initialization of Implementation 2

```
1 Rectangle** rectangles = new Rectangle * [entityCount];
2 /* ... */
3 for (int i = 0; i < entityCount; i++) {
4     /* ... */
5     rectangles[i] = new Rectangle { size, position, velocity };
6 }
```

Implementation 3 - Contiguous allocation

Implementation 3 is the same as Implementation 2 except for that the main array is composed of actual objects instead of pointers to them. This guarantees that the data is stored contiguously in memory, whereas this is not always the case in the previous implementations. The initialization is changed as shown in Listing 6.

An extra variable of 36 bytes (Figure 16) is used to simulate the frequent problem in game development, as discussed in Section 2.2.5, in which monolithic classes are used to store both variables that are used for the main calculations, as well as variables that might only be used once during the lifetime of an object. To keep the working set size the same as for the previous implementations (56 B per entity), the structure is in this implementation 16 B larger to compensate for the absence of a pointer to each entity, which is no longer needed when storing the actual values in the main array, as well as the absence of a `vpointer` in each entity, which was present in Implementation 1 - 2.

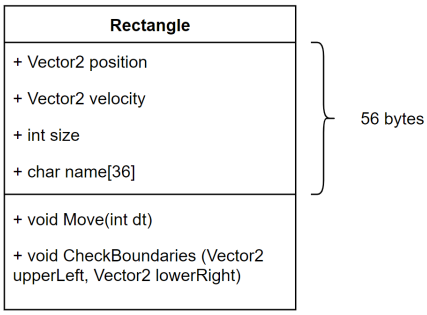


Figure 16: Class diagram for Implementation 3.

Listing 6 Initialization of Implementation 3

```
Rectangle* rectangles = new Rectangle [entityCount];
/* ... */
for (int i = 0; i < entityCount; i++) {
    /* ... */
    rectangles[i] = Rectangle { size, posX, posY, velX, velY };
}
```

Implementation 4 - Reducing the working set size

Implementation 4 was used to represent an Object-Oriented design with cache friendly locality and object size. The structures are still stored contiguously as in Implementation 3, and the game loop is kept the same (described in Listing 2), but now, the non-used variables are removed. This could have been solved in a more realistic way by storing the data not used in the loop in a separate location and referencing it by a pointer, also known as hot-cold splitting. This alternative approach is illustrated in Figure 18. But this would have needed the inclusion of an 8-byte pointer, which would have produced a class of size 32 B (28 B + 4 B added by the compiler for alignment). As the secondary goal of this Implementation was to be used as a comparison against Implementation 5, which features a Data-Oriented design with a working set size of 20 bytes per entity, the approach was instead to simply delete the unnecessary data, which generated a working set of 20 B in this implementation as well. The class layout is shown in Figure 17 below.

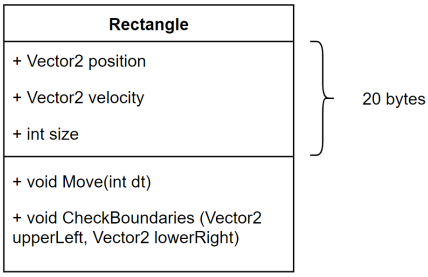


Figure 17: Class diagram of Implementation 4.

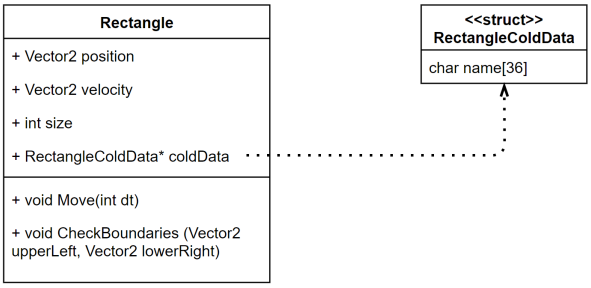


Figure 18: Alternative approach for reducing structure size (not used in the tests).

Implementation 5 - Using an Entity-Component-System design

Implementation 5 features a larger rearrangement of the program structure, inspired by the ECS pattern. The same data used in Implementation 4 still exists, but it is split into components in the form of structs, as shown in Figure 19 below. `PositionComponent`, for example, consists two floats representing the position in the game.

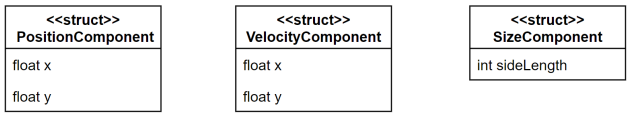


Figure 19: Components used for Implementation 5.

At initialization, instead of creating a number of Rectangle-objects, arrays of components are created and filled with values (Listing 7). The arrays have the same size as the number of entities, and each rectangle now only exists as integer, which is used to reach a specific index in each of the component-arrays. The identities span from 0 to the total number of entities (named `entityCount` in the code listings).

The main loop consists of a number of functions which operate on a limited set of component arrays. To mimic the approach of an ECS framework, they are suffixed with *-System*, but in a larger project, a System would most likely be a class with one or at most a few member functions. For this limited test though, the distinction should not matter.

The functions are run in the sequence described by Listing 8. First, `MovementSystem` uses the arrays of `PositionComponent` and `VelocityComponent`, as well as the `deltaTime` of the current iteration, to update the entities' positions. Then `BoundarySystem` uses the arrays of `PositionComponent`, `VelocityComponent` and `SizeComponent`, as well as a pointer to a `BoundaryComponent`, to check for collisions and if needed correct them. In such a small test, the memory access patterns shouldn't be all that different from what is produced with the lean, object centered design used in Implementation 4. The main difference is that the `MovementSystem` can iterate over a more packed data-set compared to how the `Move`-function operates when the entities are iterated over. The different layouts in memory are shown in Figure 20 and Figure 21.

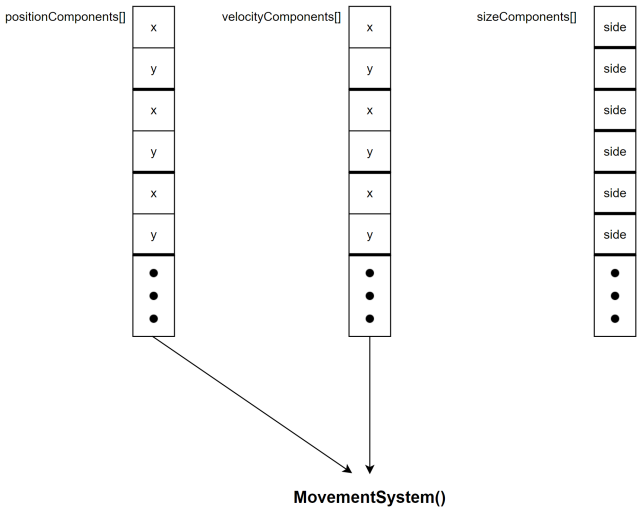


Figure 20: Data layout for the `MovementSystem`.

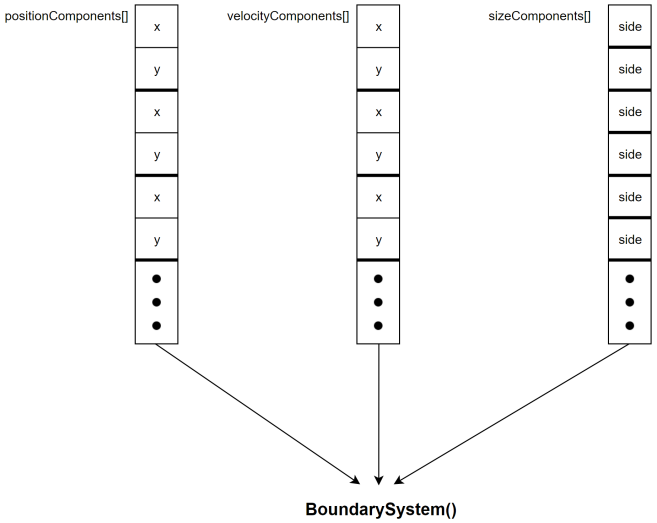


Figure 21: Data layout for the `BoundarySystem`.

The `BoundarySystem` operates on a relatively large set of data. The working set amounts to 20 B per entity, as four floats and one integer is used for each step. There was not any obvious solution to reducing this footprint. A possible alternative would have been to separate the collision detection and collision correction into two separate systems. The `BoundarySystem` could have saved the needed offset to be applied in a separate array consisting of, for example, `CollisionCorrectionComponent` and then processed them in a separate System. This is probably more in line with how a real ECS design would have been structured, as a collision probably would have needed to trigger a lot of logic such as calculating the hit-points of an entity, changing the appearance, or triggering an animation. In this example though, this would not have lessened the working set size, as the `CollisionCorrectionComponent` would have been composed of two floating point variables. Instead it would produce an increased number of computations which could have skewed the results. The benefit of the solution that was chosen, is that the number

of computations could be kept the same between all five implementations.

Listing 7 Initialization of Implementation 5

```
// Create array of PositionComponent
PositionComponent* pc = new PositionComponent[entityCount];
/* ... Fill with random starting positions ... */

// Create array of velocities
VelocityComponent* vc = new VelocityComponent[entityCount];
/* ... Fill with random starting velocities ... */

// Create array of sizes
SizeComponent* sc = new SizeComponent[entityCount];
/* ... Set size to 1 for all elements ... */

BoundaryComponent bc = { 0, 0, 1920, 1080 };
```

Listing 8 Game Loop for Implementation 5

```
while (timer.elapsedTime < targetTime) {
    /* ... Calculate time difference ... */
    MovementSystem(pc, vc, deltaTime);
    BoundarySystem(pc, sc, vc, bc);
}
```

Listing 9 Movement System

```
MovementSystem(PositionComponent pc[], VelocityComponent vc[], int deltaTime) {
    for (int i = 0; i < entityCount; i++) {
      pc[i].x += vc[i].x * deltaTime;
      pc[i].y += vc[i].y * deltaTime;
    }
}
```

Listing 10 Boundary System

```
1  BoundarySystem(PositionComponent pc[], SizeComponent sc[], VelocityComponent vc[], BoundaryComponent& bc) {
2      float dx;
3      float dy;
4      float sideHalved;
5
6    for (int i = 0; i < entityCount; i++) {
7      sideHalved = (float)(sc[i].side / 2);
8      // Check left boundary
9      dx = pc[i].x - bc.upperLeftX;
10     if (dx < sideHalved) {
11       vc[i].x *= -1; // Reverse velocity in normal direction
12       pc[i].x += sideHalved - dx; // Move back
13     } else {
14         // Check right boundary
15              dx = bc.lowerRightX - pc[i].x;
16              if (dx < sideHalved) {
17                  vc[i].x *= -1;
18                  pc[i].x -= sideHalved - dx;
19              }
20     }
21         /* Check upper and lower boundaries as in the same way as in Line 9 - 20 */
22   }
23 }
```

4.3 Test parameters and working set size

Each implementation was run on four different numbers of entities to test if the effect of an implementation change differed between different working set sizes. The working set is defined as the set of data currently worked on [6], and was estimated based on the following calculation:

$$working\ set\ size = \begin{cases} entity\ count \times compiled\ entity\ size & \textbf{if}\ iteration\ over\ array \\ entity\ count \times (compiled\ entity\ size + size\ of\ pointer) & \textbf{if}\ iteration\ over\ pointers \end{cases}$$

which builds upon the assumption that variables which are constant in size regardless of the entity count, such as the timer of iteration counter, can be neglected.

The different entity counts were based on the resulting working set size of the leanest implementations (4 and 5) in relation to the cache sizes of the hardware used for the tests. 819 entities would theoretically produce a ∼16 KiB working set thus using 50 % the L1D cache of 32 KiB. 6 554 entities would use 50 % of L2 but not fit inside L1D, and 209 715 entities would use 50 % of L3 but fit the lower levels. 838 861 entities should exceed the size of the L3 by 100 % and thus require a higher number of main memory accesses.

Table 3: Working set size of the different implementations

Entity Count	**819**	**6 554**	**209 715**	**838 861**
Implementation 1 - 3 Working set size	45 KiB	358 KiB	11 469 KiB	45 875 KiB
Implementation 1 - 3 Main method of access	L2	L3	RAM	RAM
Implementation 4 - 5 Working set size	16 KiB	128 KiB	4 096 KiB	16 384 KiB
Implementation 4 - 5 Main method of access	L1	L2	L3	RAM

For the performance tests, the main loop was run for 60 s. Then the program was exited, and the computer was let to sit idle for the same amount of time. This process was repeated 3 times, before finally calculating an average iteration count for the executions. The process was repeated for the four different test sizes and the five different implementations. The reason for spreading out the measurements was to reduce the possibility of any test being run at the same time as some unforeseen heavy background task. The time of 60 seconds was somewhat arbitrary, but it is without a doubt a time long enough as to not cloud the results with the inescapable cache misses which occur the first few loop iterations.

The base output from the test runs is the number of completed iterations. As the number of completed iterations decreases with a higher number of entities, the results are normalized against the entity count. And to enable comparison against test runs of different lengths, they the results are also normalized against the run time. This produces a metric of the number of entities processed in the game loop every microsecond, calculated by Equation 1 below.

$$entities\ processed\ per\ \mu s = \frac{iteration\ count \times entity\ count}{run\ time} \tag{1}$$

4.4 Language and compiler

The tests were written in C++ for a number of reasons. Firstly, because it is classed as a relatively low-level language. This can make it easier to estimate allocation sizes and memory locations, at least for the primitive data types. With managed languages such as Java or C#, the implementation details are often hidden under more layers of abstraction. Furthermore, C++ is the most used language in the game industry[1], [11], with notable examples such as Unreal Engine, Unity and CRYENGINE. Lastly, it is also one of the few languages with support for the Intel Instrumentation and Tracing Technology API, along with C and Fortran. This API allows for profiling specific sections of the code in the Intel VTune Profiler [48] so that unwanted parts may be filtered out.

Microsoft Visual C++ (MSVC) was used as the compiler because of its integration with the Visual Studio IDE. By default, it performs automatic vectorization of loops in the code [49]. As the ability of recognizing suitable loops may differ between the implementations, this functionality was turned off as a measure to isolate the cache utilization aspect of the tests. This was done by using the `#pragma loop(no_vector)` directive in front of each processing loop. When compiling for the `x86-64` instruction set, as was done for testing, there exists no flag to turn off auto vectorization, which warranted for this slightly more cumbersome method.

For debugging purposes, the Simple DirectMedia Layer, or SDL, library was linked to the application, which helped ensure that the physics worked as intended. But when running the tests, this functionality was deactivated by a condition flag which was checked each iteration of the loop, as well as by not initializing the SDL-engine. The reason for not including graphics when running the tests was to limit the scope of the results. The performance overhead of doing the conditional check most likely exists, but is deemed to be negligible as the number of loop iterations of the main loop is far fewer than the number of iterations over the entities. Also, any overhead stemming from this is bound to be constant between all of the tests.

4.5 Profiling

To get an estimate of how each implementation affected the hardware, the Intel VTune 2021.2 Profiler was used. This tool accesses the Performance Monitoring Unit (PMU) of the CPU through a driver to get counts of different hardware events such as cache misses and executed instructions [50]. There are many tools available to get this data but VTune was chosen as it has documented support for the Intel Core i7 processor used for testing [51].

The actual data collection was carried out by running the compiled program through the VTune GUI, and by setting it to sample the PMU counters at a 1 ms interval which is the default value. VTune can then isolate which events were caused by the tested program and estimate the absolute values of the counters which would have arisen from a complete run. By using the Instrumentation and Tracing Technology API, the program could signal the profiler to pause and resume at certain places. This allowed for limiting the measurements to the game loop by filtering out the initialization of the program. It also allowed for running multiple tests in succession without counting the hardware events happening when the tests were not running.

The profiling runs were done separately from the performance tests, but in a similar fashion to the procedure described in Section 4.3. The run time was shortened to 30 seconds, but the program was still executed on three separate occasions with a 60 s rest in between. The decreased run time was a measure to reduce the size of the VTune output which quickly reaches several gigabytes. The measurements gathered were therefore for a total running time of 90 s per type of test.

The collected hardware counters are described in Table 4 below, based on information from Intel PerfMon events for the Haswell architecture [52]. Unfortunately, no method was provided by Intel for deducing the number of cache misses for each level. Therefore, the choice of counters was based on a user guide from the Japan Advanced Institute of Science and Technology as well as information from Hadi Brais [53]–[55]. The rate of cache misses was chosen as a metric, as opposed to using the absolute value. The reasoning being that a memory bound solution could produce the same number of cache misses as a non-memory bound one when run for the same amount of time, as the number of retired instructions would also be lower. Therefore the, miss count is normalized to the number of retired instructions, a method which also used in [6]. The formulas used to calculate the cache miss rates are shown in Table 5.

Table 4: Description of PMU events for Intel Core i7 4770k.

Counter	Description
`INST_RETIRED.ANY`	Number executed instructions
`ICACHE.MISSES`	Number of instruction cache misses
`L1D.REPLACEMENT`	Number of cache lines brought in to the L1D cache causing the eviction of another line
`L2_LINES_IN.ALL`	Number of cache lines brought in to the L2 cache due to an L2 miss
`MEM_LOAD_UOPS_RETIRED.L3_MISS`	Number of memory load instructions causing an L3 miss

Table 5: Formulas for calculation of cache miss rate.

Metric	Calculated by
L1D misses per 1000 instructions	*ICACHE.MISSES / (INST_RETIRED.ANY / 1000)*
L1I misses per 1000 instructions	*L1D.REPLACEMENT / (INST_RETIRED.ANY / 1000)*
L2 misses per 1000 instructions	*L2_LINES_IN.ALL / (INST_RETIRED.ANY / 1000)*
L3 misses per 1000 instructions	*MEM_LOAD_UOPS_RETIRED.L3_MISS / (INST_RETIRED.ANY / 1000)*

4.6 Hardware

The specifications of the hardware used to run the tests is shown in Table 6 below. CPU data was collected from [9] and [56].

Table 6: Hardware used for the tests.

CPU	Intel Core i7 4770k
ISA	x86-64
Microarchitecture	Haswell
Base Frequency	3,5 GHz
Turbo Frequency	3,9 Ghz
Physical cores	4
Logical cores	8
L1D	32 KiB per core, 8-way set associative
L1I	32 KiB per core, 8-way set associative
L2	256 KiB per core, 8-way set associative
L3	8 MiB shared, 16-way set associative
Cache line size	64 B
L1 minimum access time	4 cycles
L2 access time	11 cycles
L3 access time	$\sim$ 35 cycles
Cache replacement policy	Pseudo LRU
Cache write policy	Write-back
Hardware prefetching	2 lines are fetched on a cache miss.
RAM size	16 GB
RAM speed	1600 Mhz Dual Channel

The clock frequency while testing was locked to 4 GHz as the Turbo frequency varies with temperature and power consumption [57] and may therefore induce another factor of variation for the performance tests. Additionally, Hyper-Threading (HT), which is Intel's implementation of Simultaneous Multithreading, was disabled due to some performance counters not being available in VTune with the technology enabled, and because it may lower the accuracy of the profiler output [58]. But since the tests are single threaded, the execution of the program should not be affected in one way or another by the technology. Theoretically, disabling it could produce results that are less affected by background threads, since some conditions, not controlled by the test, may make a background thread take advantage of HT, thus letting it finishing quicker and leaving more resources to the thread running the test.

5 Results and analysis

In the following section, the collected experimental data is presented in a condensed form and analysed.

Section 5.1 presents the findings in the form of performance results and collected hardware data for all five implementations when tested on four different numbers of entities.

Then, Sections 5.2 to 5.5 put forward four narrowed comparisons, each of which analysing the effect an isolated change in the implementation has on performance and cache utilization. A summary of the comparisons is given in Table 7 below.

Table 7: Summary of the comparisons made to produce the results.

Comparison	**Intent**
Implementation 1 & 2	Observe the effect of removal of runtime polymorphism
Implementation 2 & 3	Observe the effect of going from non-contiguous to contiguous allocation
Implementation 3 & 4	Observe the effect of reducing the working set size
Implementation 4 & 5	Compare an OOD approach implementing the three optimizations above to an equivalent ECS-design

5.1 Overview

The performance results of all the tests are presented in Figure 22 below, where the y-axis shows the number of processed entities per microsecond - a metric which is inversely related to the CPU time necessary for running an implementation for a fixed number of iterations.

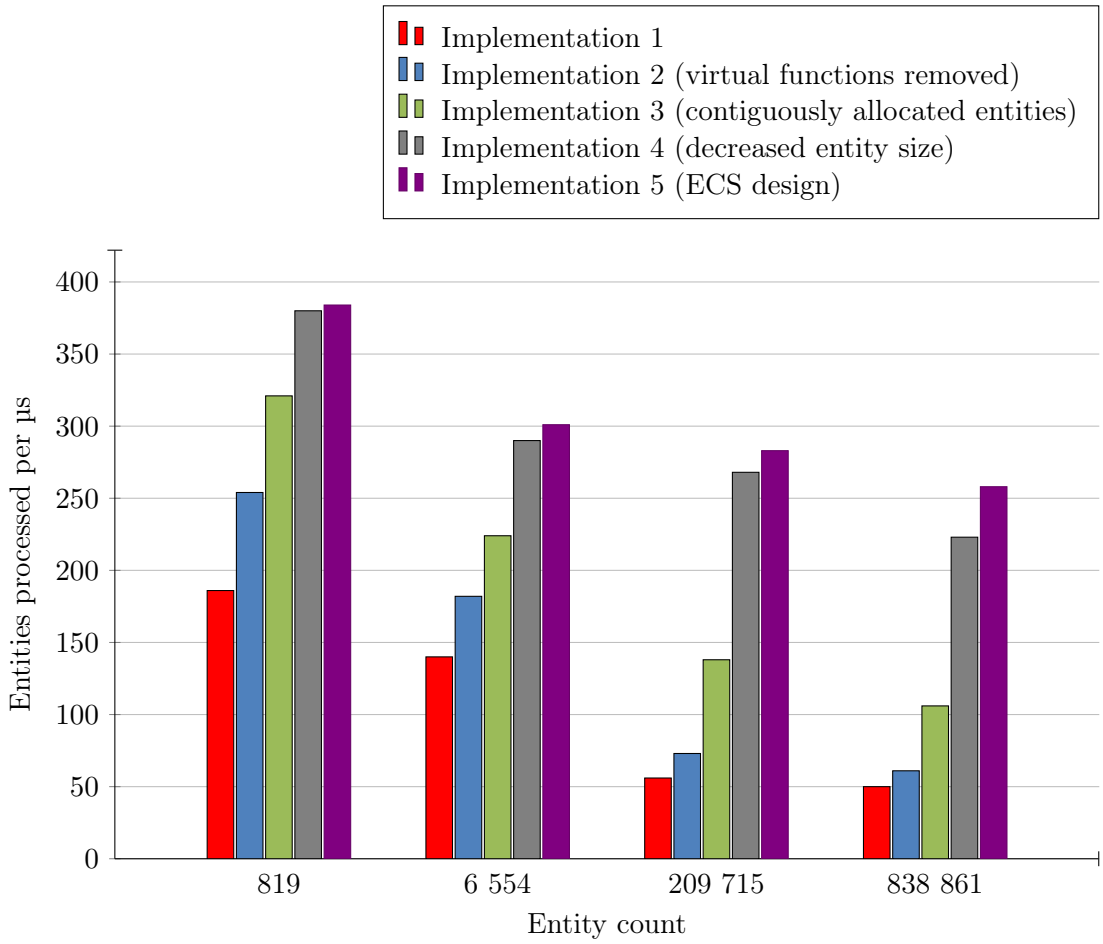


Figure 22: Entities processed per microsecond for all tested implementations and entity counts.

As is visible, all implementations fare worse when the number of entities increase. Intuitively, the computational overhead of running the game loop should decrease when the entity count increases. The reasoning being that more entities to process per iteration should result in a lower number of total iterations in a fixed time, which reduces both the amount of loop related instructions as well as the overhead from time measurements. But the real-world performance metrics tell another story. The average number of processed entities for all implementations actually decreases by 60 % when comparing the lowest entity count to the largest. This increase can't be explained by the algorithm moving the entities and calculating collisions, as it operates in $\mathcal{O}(n)$ complexity and the results are normalised to the number of entities. This indicates that something other than computational power sets the limit when the working set increases.

It is also apparent that there exists a large difference between different implementations, even though they perform the same task with the same number of computations for each entity. A difference that is also widened as the number of entities is increased. At the lowest number of entities, the performance difference between Implementation 1 and Implementation 5 is 206 %, while at the largest number, it has grown to 514 %.

In Figure 23 to 26 below, the collected hardware data is presented in the form of cache misses per 1000 instructions (MPKI). From Figure 24 to 26 it can be derived that when the working set of an implementation goes from being able to be completely contained in a certain level of data cache to exceeding it, the rate of cache misses rises, but is then relatively constant when the working set is increased further. Seeing that both Implementation 1 and 2 follow this trend, there must still be a certain level of continuity in the way the entities are located even though this could not be guaranteed by the method of allocation used. It is also apparent that since the working set of Implementations 1 - 3 is too large to fit in the L1D cache even at the lowest number of entities, the mentioned rise in cache miss rate is absent for that particular level.

The pattern of miss rate of the dedicated instruction cache (L1I) is harder to interpret. In general, there seems to be an increase in L1I miss rate for all implementations when the entity count increases, but this common trend is hard to explain with the help of the presented background. The miss rate is significantly lower than for the L1D and the unified caches, so it might simply be that the test program consists of too little logic, and in turn machine instructions, to produce any meaningful differences between implementations and test sizes.

There also seems to be an inverse correlation between how well a certain implementation performed and the miss rate for all cache levels, which indicates that the increase in performance comes from the decreased memory latency, as hypothesized. The noteworthy exception is Implementation 2, which did better than Implementation 1 in terms of performance, but where the cache metrics do not reflect this. This suggests that another underlying mechanism than the cache is responsible for the decrease in CPU time when virtual functions are removed.

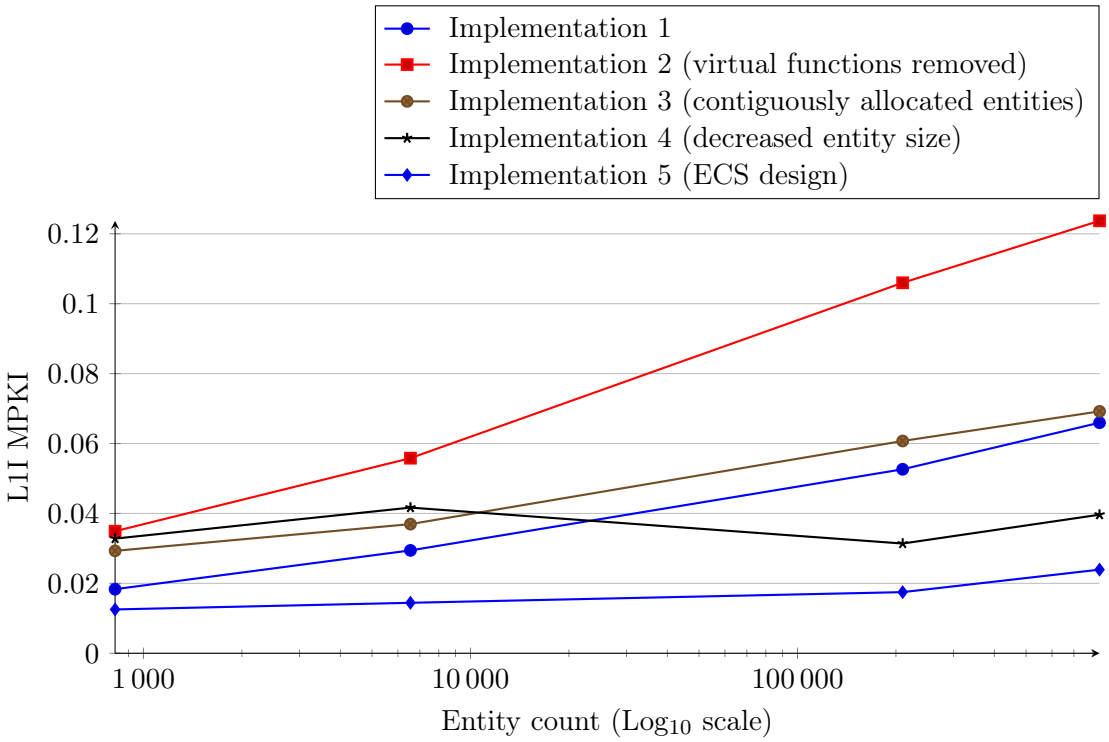


Figure 23: L1I misses per 1000 instructions for all tested implementations and entity counts.

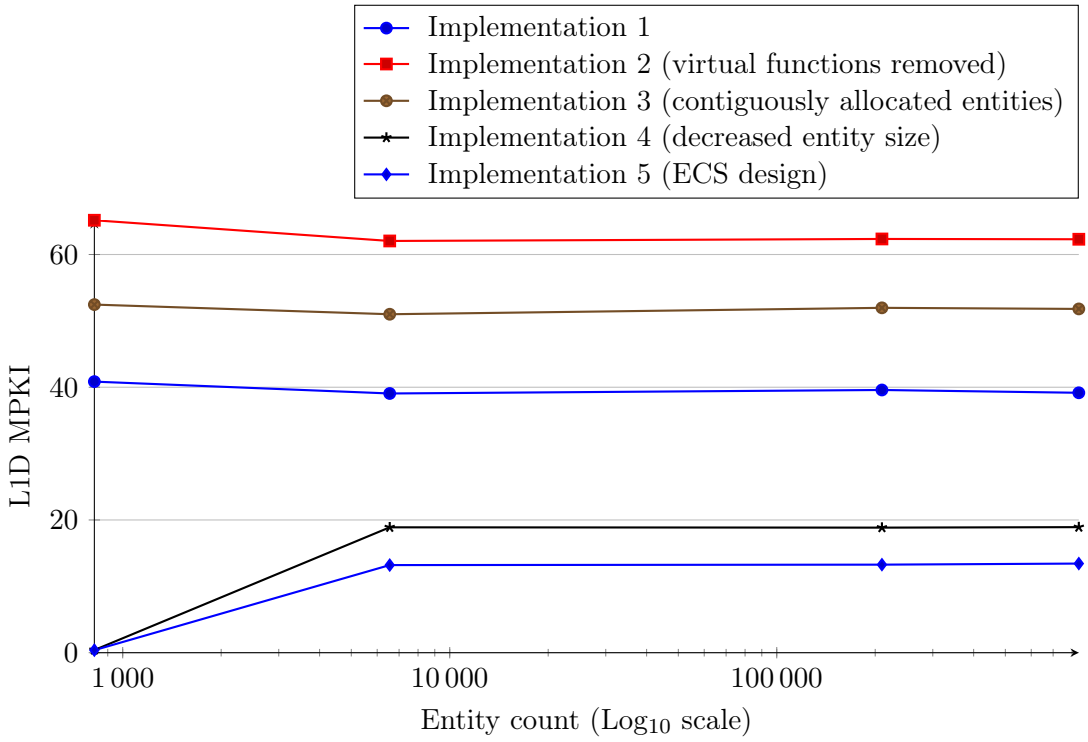


Figure 24: L1D misses per 1000 instructions for all tested implementations and entity counts.

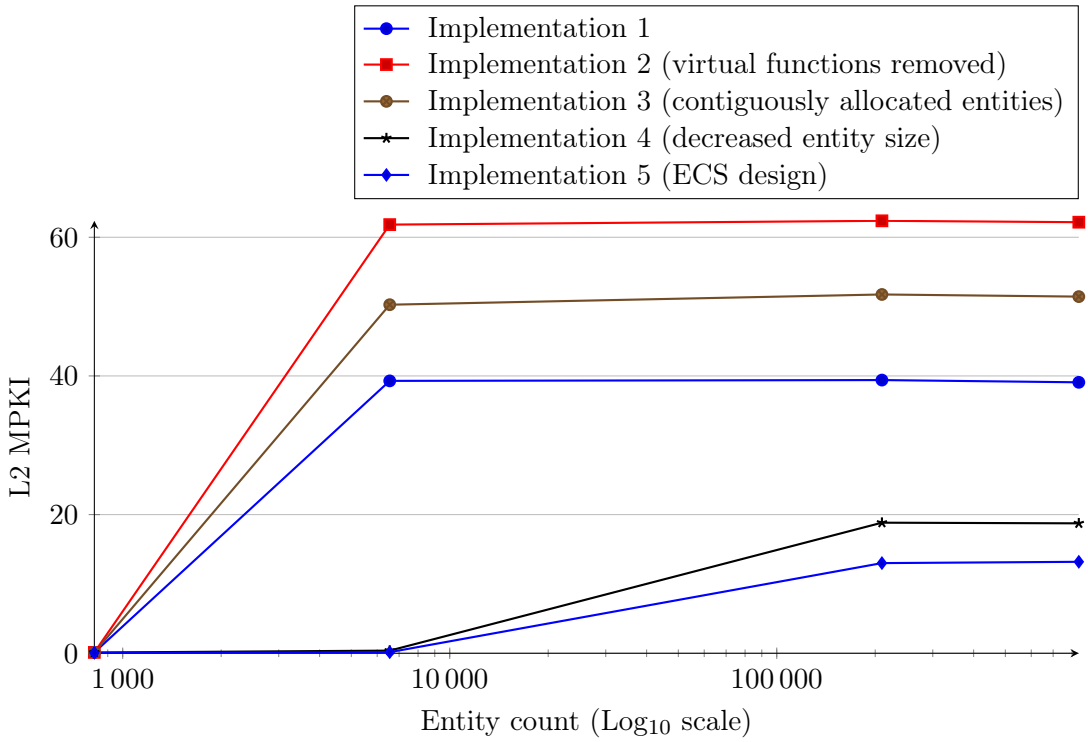


Figure 25: L2 misses per 1000 instructions for all tested implementations and entity counts.

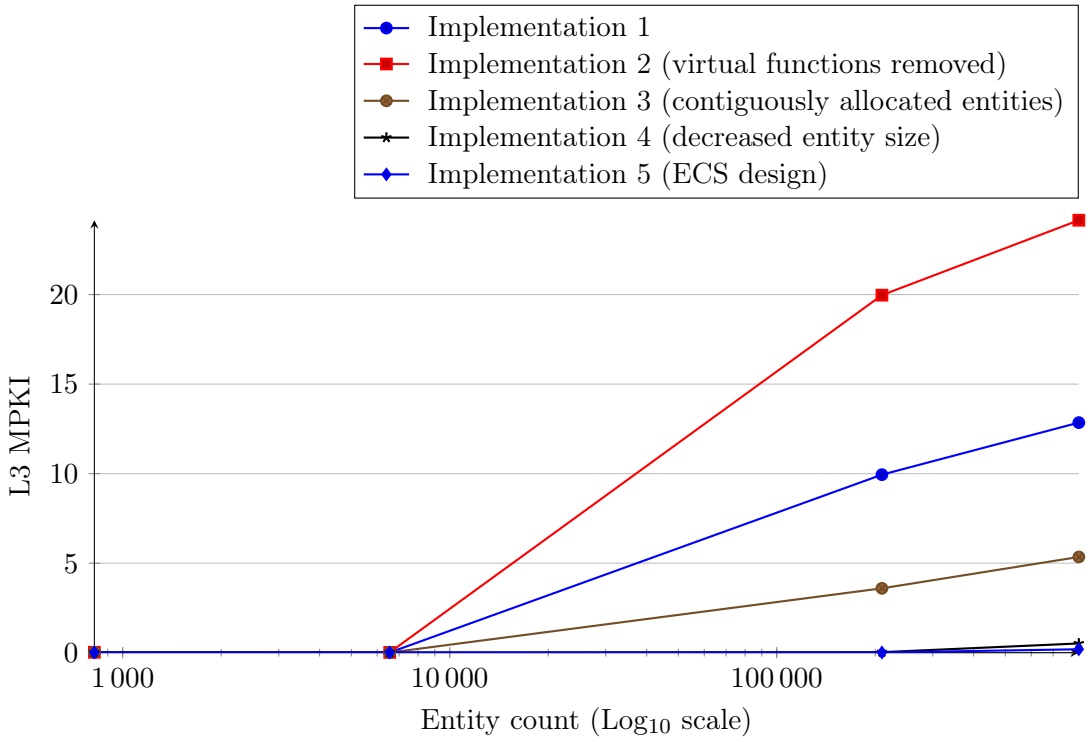


Figure 26: L3 misses per 1000 instructions for all tested implementations and entity counts.

The results are further broken down in the following sections where the focus lies on the specific change between two implementations.

5.2 Removal of run-time polymorphism

The performance difference obtained by removing two virtual function calls per loop iteration is shown in the comparison in Figure 27 below. Implementation 1 features two virtual calls per loop iteration, while they are removed in Implementation 2. The difference is most pronounced at 819 entities, where it amounts to a 37 % increase in processed entities. When increasing the entity count, the gap narrows to about 23 %.

The results show that there certainly is some overhead to the virtual calls. However, it is not explained by L1I misses as presumed, which could occur when an extra indirection is added to reaching the function addresses. Nor is it explained by an increase of data cache misses at any level which could occur due to the extra burden of keeping the `vtable` in the cache. Implementation 2 instead has a higher rate of misses across the board. One possible explanation is that for such a small set of functions the L1I is still large enough not to be affected, while the size of the `vtable` is simply too small to have a visible effect on the other caches.

The, non-intuitive, lower, miss rate for Implementation 1 might be caused by the inability of the compiler to inline the `Move` and `CheckBoundaries` functions, and possibly unroll the main loop, which would produce machine code with a smaller footprint. The higher performance for Implementation 2 is then only due to the lessened overhead of not having to dereference the `vpointer` and not doing two lookups in the `vtable` for each iteration over an entity, as well as the possibility of more inlining and unrolling which may further decrease the cycle cost per iteration. Also, there is the possibility that part of the performance difference is explained by the use of the `vtable`. As the table resides in the data-cache, it always faces the possibility of being evicted as even the lowest entity count exceeds the size of L1D. This event would likely not be visible in terms of miss rate but could induce further delays for each function call done in Implementation 1.

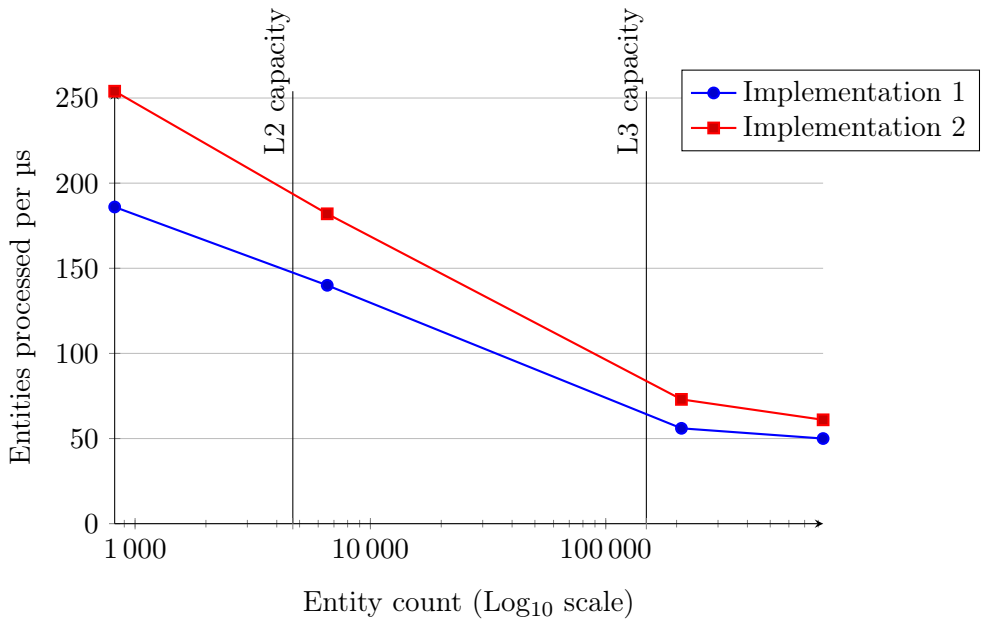


Figure 27: Performance when using virtual functions (Implementation 1) vs. when using regular functions (Implementation 2).

5.3 Contiguous allocation

When going from allocating the entities with the `new`-keyword, and thus not guaranteeing a contiguous storage (Implementation 2), to instead allocating them in an array (Implementation 3), performance improves by 23 % to 90 % with a larger relative difference at the two higher entity counts. The performance comparison is shown in Figure 28.

Examining the cache, the miss rate of the instruction cache is higher for Implementation 1 in the two lower sized tests, and then goes down to about 90 % of the rate for Implementation 2. Theoretically, there is no reason to why it should differ between the tests, so it might be fluctuations caused by background processes, or to some unforeseen optimization by the compiler. The L1D miss rate is close to constant for both implementations across all sizes, with an 17-18 % lower rate for Implementation 3. The L2 miss rate is also about 17 % lower for implementation 3, but with a low number of misses for both implementations when the working set fits inside L2 ($<$ 6554 entities). In L3 the same spike in cache misses can be discerned for both implementations as soon as the working sets exceeds it. The miss rate is about 80 % lower, though, for Implementation 3.

As both implementations feature the same working set size, it is natural to see these changes in miss rate appear at the entity counts, due to an increase in capacity misses. The difference between them is likely due to a higher number of conflict misses for Implementation 2 as the more spread out data spans a larger set of memory addresses. It can likely also be explained by a rise in compulsory misses as the prefetcher will not always be able to detect a pattern in accesses to the non-contiguous layout.

Since the method of access differs between the two implementations - the entities of Implementation 2 being referenced by pointers while in Implementation 3 they accessed

as elements in an array - one cannot for sure say that it is not the overhead of the extra pointer dereferences that causes, or at least contributes to, the lower performance. It is unlikely to have a large effect though, as the pointers are stored contiguously, and will with high probability continually be prefetched into L1D without any large effort, in a stream separate from the entity data.

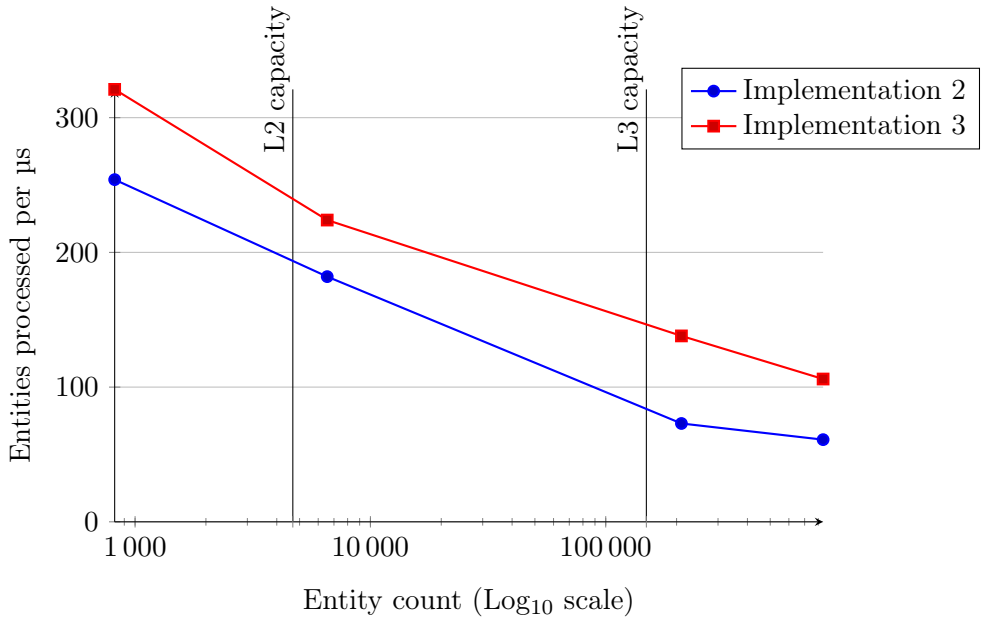


Figure 28: Performance when iterating over pointers of non-contiguous entities (Implementation 2) vs. when iterating over entities contained in an array (Implementation 3).

It is to be noted that the circumstances of the comparison were likely nearly optimal for the non-contiguous implementation. The first reason being that the ”wanted” data is all located at the beginning of the structure. When not explicitly aligning each structure to the beginning of a cache line, one cannot be sure that the data needed for a function is all located in the same line. The probability of it being split into two lines is in this implementation kept to a minimum as the data is bunched up, as mentioned.

Imagining that each structure is aligned to the start of a cache line, then if the size was smaller, each access to a disjoint entity would have caused other, non-relevant data to also be fetched. The same would also have been true if each structure was only a bit larger as to span two cache lines, and if the needed members were in different lines. Then each access would have fetched one line of mostly "wanted" data, and another with a low fraction of usable data.

For the contiguous case, the above matters would not have had as large of an effect. If data spanned two lines, or only a fraction of one line, then the remainder of the line of low utilization would have been composed of members belonging to the next entity (or entities) in the array, which would soon be needed and fetched by the CPU anyway.

Secondly, the allocation of entities is done in a linear loop at the start of the program.

This will likely place them in a semi-contiguous block of memory, as the heap will not yet have become fragmented by any meaningful degree. Other studies, as one made by [59] have shown large increases in access time when introducing "garbage data" in between allocations, as well as when the entities are dynamically added and removed during runtime. Löfstedt also observed this phenomenon as a decrease in performance when memory fragmentation was increased by adding and removing entities during runtime [45].

5.4 Reducing structure size

In a design already featuring iteration over a contiguous data set, the comparison between Implementation 3 and Implementation 4 shows the performance difference when reducing the size of the data. Each entity is reduced to 20 B, which equals a reduction in the working set by 36 B per entity. This allows for more useful data to fit in each level of cache, as well as getting more use of each fetched cache line. As shown in Figure 29 below, the performance is about around 20 % higher for Implementation 4 at the lower entity counts, increasing to 100 % at the two higher entity counts.

The cache miss rate for L1D and the higher cache levels all follow the same pattern. When the larger working set of Implementation 3 goes outside the span of a certain cache level, Implementation 4 will still be contained. This creates a spike in misses for Implementation 3 with a large gap between two until Implementation 4 also exceeds said level. When both implementations fit in the same level, e.g. L2 at 819 entities, and L3 at 6 554 entities, the cache miss rate is in that cache level low for both implementations.

The misses are most likely constituted by capacity misses, and not conflict misses, by virtue of the high locality of both implementations. Possibly, some of the difference in miss rate can be attributed to compulsory misses if the lower cache line utilization for Implementation 3 causes the prefetcher to not be able to keep up the pace.

Curiously the L1I miss rate is slightly higher for Implementation 3 at the two lowest entity counts, but then falls to a level of around 50 % for the higher counts. This anomaly is difficult to explain with the given background, as no mechanism explains this behaviour. Therefore, it warrants for more careful inspection. But one should at the same time be aware that the absolute value of the miss rate is at a very low level compared to for example the L1D miss rate, so perhaps the difference is due to errors in the measurement or other background noise.

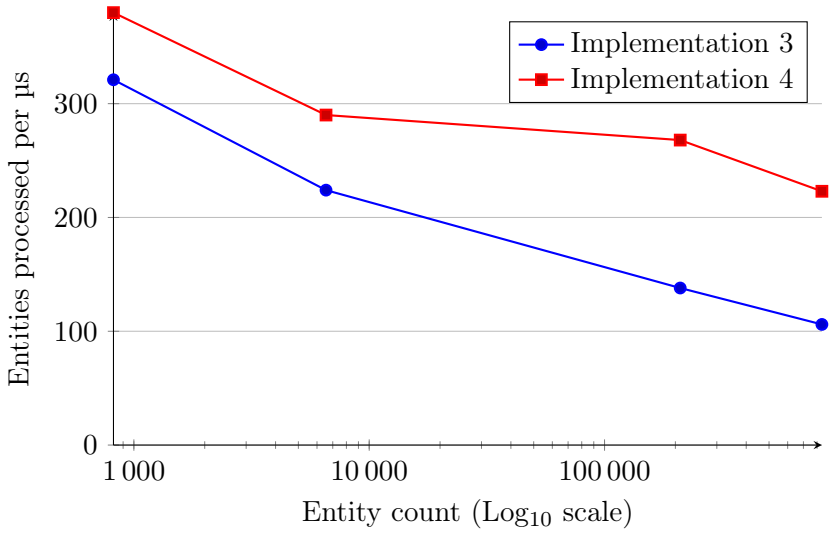


Figure 29: Performance when iterating over entities of size 56 B (Implementation 3) vs. when iterating over smaller entities of size 20 B (Implementation 4).

5.5 Using an Entity-Component-System design

Implementation 4 features a design which adheres to the examined Data-Oriented principles, namely no virtual functions, iteration over a contiguous collection of entities, and a low entity size. Implementation 5 is also built upon these principles but uses an Entity-Component-System design instead of the Object-Oriented design used in Implementation 1-4. The result is a slight rise in performance, which becomes more defined exceeding 209 715 entities, amounting to a 6 % difference, and finally reaching a 16 % higher number of processed entities per time unit at the maximum test size.

A lower rate of L1I misses (around 50 %) can be observed for Implementation 5. The absolute number of L1I misses is low for both implementations, so perhaps too much should not be read into the difference. Since the actual calling pattern and the operations inside the functions are mostly the same in both implementations, this is likely not a factor. The lower miss rate for the ECS design could simply be due to the compiler being able to produce a smaller output, which then has a lower probability of being evicted.

Furthermore, Implementation 5 shows a ∼70 % lower rate of L1D and L2 misses and a 35 % lower L3 miss rate when the working set exceeds each specified level of cache. As seen in previous comparisons, when the working set fits in a certain level of cache, the rate of misses is low for both implementations.

The data size is the same between the implementations. Implementation 4 has a structure size of 20 B, while the largest System (function) in the ECS design, which is the `BoundarySystem`, also uses 20 B of data per entity. It is therefore not a surprise to see a rise in cache misses for both implementations occurring at the same entity counts - when the total amount of data exceeds a certain cache level. The difference, which is more or less constant for each level, can likely be connected to way the smaller `MovementSystem` accesses the data, compared to the `Move`-function used in Implementation 4.

The `MovementSystem` only touches the array of positions and velocities. The `Move`-function on the other hand, while in the same way only needing the position and velocity members, also causes the size-member of each entity to be present in the fetched cache lines. This theoretically lowers the utilization by about 20 % (4 B of non-used data divided by 20 B of total data per entity) for the Move-step of the loop. The situation is not as bad as it could be, since the following calls to the `BoundaryCheck`-function are in need of the `size`-member which may then be reused. In large entity counts though, there is a good chance that it will already have been evicted, which may be an explanation for the amplified performance difference in favor of Implementation 5 seen when the working set is increased.

Another possible mechanism behind the difference in performance and cache miss rate is that the ECS implementation keeps read and write operations better separated when looking at a cache line level. The `Move`-function will change the position, but only read the velocity. Each function call will, due to the write, mark the entire cache line as dirty, which might potentially force a RAM-write for approximately every fourth Move-call (since three entities of 20 B will be present in each 64 B cache line). The `MovementSystem`, on the other hand, will have a set of cache lines consisting of only positions, and another set of only velocities, which forces a write for every 9th processed entity (8 positions of 8 B will fit inside a 64 B line). The same difference in write behaviour is true for the collision detection stage, but possibly being of lower advantage for the ECS design since the velocity will also be written to in some of the iterations (when an entity collides with a boundary).

It is also notable that both implementations suffer the most, in terms of CPU time, when the working set goes outside the span of L1D. This yields a relative decrease in performance of $\sim$22 % for both of them, which can be compared to the $\sim$8 % decrease when going from the L2 to the L3. This trait could indicate that the cycle cost of the actual calculations is relatively low. Otherwise, the rise from a 4 cycle cost for an L1D memory fetch, to an 11 cycle cost for the same operation on L2 (as specified by Intel), would reasonably be masked by the cycle cost of performing calculations on the three entities fitting in a cache line.

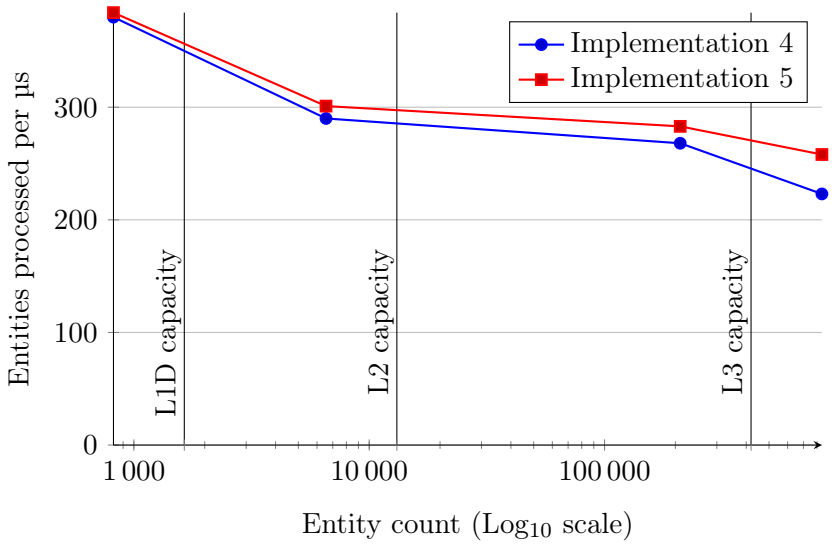


Figure 30: Performance for the ECS design (Implementation 5) vs. a cache friendly Object-Oriented design (Implementation 4).

6 Discussion

This chapter summarises the findings of the thesis. Section 6.1 discusses the relation of the results to the purpose of the thesis and to previous work done in the area. Section 6.2 then reasons about the validity of the results due to the limitations of the methods used and the scope of the thesis. Section 6.3 presents the takeaway of the thesis and some of its implications. Finally, in Section 6.4, further work that could be done in the area is suggested.

6.1 Interpretation of results

The thesis set out to answer the question of how restructuring code in accordance to Data-Oriented Design principles affect CPU time and cache utilization, and how the result is applicable in the field of video game development. The answer is that performance, in terms of CPU time, is affected in a positive way for all the examined changes. The effect on cache utilization is more complicated, however. A lower cache miss rate for the data and unified caches could be observed when following DOD guidelines, while the rate of instruction cache misses was low altogether. An exception to the lower observed cache miss rate was when observing the effect of removal of run-time polymorphism, which is discussed further below. Since the background showed that DOD principles are often deviated from in the area of game development, the results are likely applicable in this field, which is further discussed in Section 6.1.2.

Removal of run-time polymorphism

The first experiment performed, which looked at the removal of run-time polymorphism in the form of removing virtual function calls, did show a reduction in CPU time. But at the same time, the rate of cache misses increased slightly. This indicates that the gained performance did not stem from better cache utilization but instead from the lower CPU-overhead of using regular functions, as well as increased freedom for the compiler to optimize the code. The absence of the speculated decrease in miss rate can likely be explained by the small scale of the experiment. Only a single class with two different virtual functions were used. A larger selection of derived classes iterated over in a possibly unsorted container would likely have increased the miss rate. Additionally, a larger program would generally produce a greater number of instructions, which in turn would put more strain on the limited instruction cache.

Contiguous allocation

When observing the effect of going from iterating over a possibly non-contiguous set of entities, to instead iterating over an array, the result was a decrease in CPU time as well as a lowered cache miss rate. The mechanism behind the changes is likely a combination of a decrease in conflict misses as well as allowing for more effective prefetching. The performance effect was modest, but the test conditions were likely favorable for the non-contiguous implementation, which means that a larger difference can generally be expected.

Reducing structure size

Reducing the entity size from 56 bytes to 20 bytes did also result in a decrease of the required CPU time per processed entity, as well as a decrease in the cache miss rate, which is in line with the theoretical background which explains the benefit of keeping a high fraction of hot data in the caches. It also shows the that there can be a large performance difference when such a change enables for the working set to be kept in a lower level of cache, and not just when it lessens the number of RAM accesses.

Using an Entity-Component-System design

The last experiment, comparing an OOD to an equivalent ECS design, showed a slight edge for ECS regarding both performance and cache miss rate, with an increasing difference when the number of entities was increased. This shows that an Object-Oriented design adhering to DOD principles can perform relatively close to an ECS design, but that the latter might be necessary to reach even further. If a larger scale test was conducted, however, it would likely be increasingly difficult to keep cache utilization high using an OOD, which could produce a larger difference between the approaches.

6.1.1 Summary

In the performed experiments, decreasing the structure size had the greatest effect on performance with a 110 % performance increase at the largest number of entities. But with so many factors that can differ between an isolated test and a real-world application, such as the function design, ordering of variables, call order, and introduction of other logic and data, not to mention running the tests on other hardware, it is likely not meaningful to classify the changes after the magnitude of their effect. The results do, however, show that each change yielded better performance, so it is not unwarranted to claim that adhering to Data-Oriented principles can be beneficial for performance. This statement is also in line with the findings of Faryabi [43], Nilsson & Björkman [44] and Löfstedt [45] which show that adhering to Data-Oriented principles in the program design can lead to better performance, especially when processing large amounts of data.

While the scope of the thesis limits the experiments to only look at select parts of what constitutes a Data-Oriented Design, it is still apparent that the examined principles seem to affect the cache utilization in the form of a lower cache miss rate for the data and unified caches, apart from the principle of removing run-time polymorphism. It is not proven that a decrease in cache miss rate is the direct cause of the decrease in CPU time, but there at least seems to be a correlation, with the removal of run-time polymorphism being the exception as mentioned. The rate of instruction cache misses is low for all implementations, which is likely a result of the small size of the test-program. The lower data cache miss rate is consistent with the findings of Löfstedt, who also observed a decrease in L3 cache miss rate when implementing a game engine following Data-Oriented principles [45].

6.1.2 Applicability in the area of game development

Since the experimental approach was to implement a single isolated part of a game engine, care must be taken when extrapolating the findings to a larger scale game. The

theoretical background shows that game engines often have a lot of untapped potential in the form of cache utilization, and earlier works such as [43] do show that a Data-Oriented Design may also benefit a fully implemented game. So, there is reason to believe that the results are also valid when increasing the scale. As mentioned, a dynamic system increases the memory fragmentation, which could lead to more room for improvement regarding the cache. This was a characteristic also observed by [45] when performance and L3 cache misses were examined on a system with a varying degree of dynamic behaviour. The larger set of logic and data which comes with a large game would also put more strain on the caches, adding room for improvement. In these experiments, the rate of instruction cache misses was consistently at a very low level due to the low amount of program logic. Therefore, a full-scale game could probably see more benefit in optimizing for the instruction cache.

This report excluded any rendering of graphics from the tests as to keep focus on the CPU and also reduce the additional variables which could be generated by using a third party graphics library, but works such as [43] and [44] have shown DOD to beneficial in these cases too. One could expect there to be a limit to what can be achieved in terms of performance by improving cache usage if the program is bound by the GPU. But this should be the case for any strategy for decreasing CPU time, and not just for strategies targeting the cache.

Regarding the operation cost of the algorithms that might be part of a larger game, one could speculate that if many high cost operations are involved, such as ones involving square root calculations or other cycle heavy instructions, this could in some cases mask the cost of a cache miss in the lower cache levels by giving the prefetcher more time to ready the next needed values. Therefore, parts of an engine using such operations might see a limited gain from improving the cache utilization. Although, if the values needed are spread out too much in memory such that the prefetcher cannot detect the access pattern, the cost of slow memory accesses would instead be added to the already long running calculations.

The experiments conducted in this paper used algorithms with $\mathcal{O}(n)$ time complexity, whereas $\mathcal{O}(n^2)$ or $\mathcal{O}(nlog(n))$ are not an unusual sight in for example the collision detection algorithm used in some games. A hypothesis is a higher time complexity could further amplify the performance gain achieved by keeping a larger fraction of hot data in the caches. The rationale being that such algorithms often feature a high temporal locality since they loop over the same data multiple times. Nilsson & Björkman examined such an algorithm, in the form of comparing a $\mathcal{O}(n^2)$ collision detection algorithm implemented in an OOD to one implemented using an ECS design. The ECS design showed only slightly better performance exceeding a working set of 140 KiB, but the results are at the same time somewhat hard to relate to since the implementation details of the OOD are not disclosed, and the ECS based design is implemented using the external EntityX-framework. Also, the tests are rendered, which might set the GPU as the limiting factor.

6.2 Limitations

As the time available was finite, some limitations had to be made to keep the scope of the thesis. The main limitation was to restrict what parts of the Data-Oriented Design should be examined. Since the mechanism that is often held central to its effect on

performance is its beneficial effect on the CPU cache, principals related to this were chosen as the main focus. A further narrowing of the topic had to be made, which was to only examine the effect on single threaded execution.

Furthermore, the experiments could only look at a select few ways of optimizing cache usage. Software prefetching is not investigated, nor is the effect of separating reads and writes, other than indirectly, when the ECS pattern is examined. Also, no consideration is taken of the alignment of data with regard to cache lines, which could have made a change for the comparison between non-contiguous and contiguous access, as is discussed in Section 5.3. Therefore, there are still many cache related aspects of Data-Oriented design which are left unexplored in this report.

The main limitation of the experimental approach is that the execution was very specific. They tests were all performed on a single machine, implemented in the same language, and compiled with the same compiler. The results could likely vary between languages depending on their underlying implementation. The compiler also has a lot of power in optimizing the output machine code, which adds the factor of how this is handled in different compilers. Also, as the architecture of different CPUs varies widely, with different cache sizes, levels of associativity, instructions costs and feature sets which could have an effect on the potential performance benefit of optimizing for the cache.

Since the tests were not run in an isolated environment, but on a desktop machine with various installed software, different background processes might affect the CPU by a varying degree at different times depending on a myriad of uncontrollable factors. Although the tests were performed at different times with a rest period in between, and on a freshly rebooted computer with no user interaction or foreground processes, the gathered results may still contain inaccuracies as an effect of this.

6.3 Conclusion and recommendations

The takeaway of this report is that designing a program with Data-Oriented Design principles in mind can have a favourable effect on performance. This might be especially relevant in the area of game development as the theoretical background indicates that the non-DOD endorsed practices of using polymorphic functions, pointer iteration and monolithic objects are commonplace in this particular field. The implication is therefore that there is reason to examine the implementation at data level, and not only at an algorithmic complexity level, when designing a game engine.

The first recommendation would be that if one uses pointer based data structures liked linked lists simply by habit, rather than necessity, or due to reasoning that its lookup time complexity is adequate for use in a hot loop, switching to a contiguous data structure may be advantageous for performance.

Also, abandoning polymorphism may increase performance by lowering the overhead of the virtual function calls, but also by no longer hindering the use of an array. If the use of inheritance is abandoned altogether, this act may also limit the amount of ”unnecessary” data being part of the derived classes, which may aid the cache utilization and, by implication, performance.

In the same spirit, other ways of reducing data that is seldom used should also be

considered, such as hot/cold splitting. The choice of smaller data types could also have an impact, and should not be neglected on the premise that memory is available in abundance. Choosing a short int over an integer, or a float over a double where the extra precision is not needed, could have a significant impact on the utilization on a cache line level. It may also be worthwhile to look analyse the placement of said variables inside a class or struct, and try to keep members that are used together in the program logic also placed in close vicinity inside the structure, while also trying to avoid unnecessary padding added by the compiler by having the members sorted by size. Both strategies will be beneficial for the cache, and likely also for CPU performance.

Finally, if a conversion to the ECS pattern is feasible for a certain program architecture, it should be considered, as it may allow for an even more efficient use of the cache than what is possible with an Object-Oriented Design, even if both are built upon Data-Oriented principles.

6.4 Future Work

Since the result of removing virtual function calls was somewhat unexpected, it would be interesting to analyse the assembly output to investigate if an added amount of inlining is the answer to the increased cache miss rate. Also, it would be interesting to implement a multi-threaded test suite and observe how the Data-Oriented Design principle affects performance when execution on multiple cores is introduced, since easier parallelization is one of the main selling points of the DOD-based Entity-Component-System.

References

[1] J. Gregory, *Game Engine Architecture*, 3rd ed. US: Taylor & Francis Group, 2019.

[2] R. Fabian, *Data-Oriented Design: Software engineering for limited resources and short schedules*, 1st ed. US: Richard Fabian, 2013.

[3] E. Blem, J. Menon, and K. Sankaralingam, "A detailed analysis of contemporary arm and x86 architectures," *UW-Madison Technical Report*, 2013.

[4] A. Fog. (2021-03-22). "Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD, and VIA CPUs," Technical University of Denmark, [Online]. Available: `https://www.agner.org/optimize/instruction_tables.pdf` (visited on 2021-05-18).

[5] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 5th ed. US: MK Publications, 2012.

[6] U. Drepper, *What Every Programmer Should Know About Memory*, Red Hat, Inc., 2007-11-21.

[7] D. Levinthal, *Performance Analysis Guide for Intel® Core™ i7 Processor and Intel® Xeon™ 5500 processors*, Intel Corporation, 2009.

[8] C. Carruth, *Efficiency with Algorithms, Performance with Data Structures*, Paper presented at the CppCon 2014 conference, Bellevue, 2014-09-08. [Online]. Available: `https://github.com/CppCon/CppCon2014/tree/master/Presentations/Efficiency%20with%20Algorithms%20C%20Performance%20with%20Data%20Structures` (visited on 2021-05-06).

[9] P. Hammarlund, *4th Generation Intel Core Processor, codenamed Haswell*, Intel Corporation, 2013-08.

[10] J. Thiyagalingam, O. Beckmann, and P. H. Kelly, "An exhaustive evaluation of row-major, column-major and morton layouts for large two-dimensional arrays," in *Performance Engineering: 19th Annual UK Performance Engineering Workshop*, 2003, pp. 340–351.

[11] J. Blow, "Game Development: Harder Than You Think," *ACM Queue*, vol. 1, no. 10, pp. 28–37, 2004.

[12] Oracle Corporation. (2021-03-01). "The Java Language Specification, Java SE 16 Edition: Chapter 10. Arrays," [Online]. Available: `https://docs.oracle.com/javase/specs/jls/se16/html/jls-10.html` (visited on 2021-05-19).

[13] Microsoft Corporation. (2017-01-07). "C# language specification | Microsoft Docs," [Online]. Available: `https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification` (visited on 2021-06-03).

[14] W. Cohen. (2016-07-01). "How to avoid wasting megabytes of memory a few bytes at a time," Red Hat Inc, [Online]. Available: https://developers.redhat.com/blog/2016/06/01/how-to-avoid-wasting-megabytes-of-memory-a-few-bytes-at-a-time (visited on 2021-06-01).

[15] A. Kleen, L. Mircea, I. Peled, E. Steinberg, and M. Anschel. (2007-10-29). "Optimizing for instruction caches, part 1," Electronic Engineering Times, [Online]. Available: https://www.eetimes.com/optimizing-for-instruction-caches-part-1/ (visited on 2021-05-26).

[16] B. Stroustrup. (2012-10-03). "Bjarne Stroustrup's C++ Glossary," [Online]. Available: https://www.stroustrup.com/glossary.html (visited on 2021-05-12).

[17] K. Driesen and U. Hölzle, "The Direct Cost of Virtual Function Calls in C++," vol. 31, 1996-10, pp. 306–323. DOI: 10.1145/236338.236369.

[18] I. Bogosavljević. (2021-02-21). "The true price of virtual functions in C++," Johny's Software Lab, [Online]. Available: https://johnysswlab.com/the-true-price-of-virtual-functions-in-c/ (visited on 2021-05-19).

[19] P. Artoumas. (2019-04-01). "Static and Dynamic Dispatch," [Online]. Available: https://medium.com/ingeniouslysimple/static-and-dynamic-dispatch-324d3dc890a3 (visited on 2021-06-01).

[20] B. Stroustrup, *The C++ Programming Language*, 4th ed. US: Pearson Education, Inc., 2013.

[21] A. Fog, *Optimizing software in C++*, Technical University of Denmark, 2021.

[22] S. Brand. (2020-03-02). "The Performance Benefits of Final Classes," Microsoft Corporation, [Online]. Available: https://devblogs.microsoft.com/cppblog/the-performance-benefits-of-final-classes/ (visited on 2021-05-12).

[23] H. Sutter and A. Alexandrescu, *C++ Coding Standards: 101 Rules, Guidelines, and Best Practices*, 1st ed. US: Pearson Education, Inc., 2005.

[24] J. M. L. Muñoz. (2021). "An efficient polymorphic data structure," [Online]. Available: https://www.boost.org/doc/libs/1_76_0/doc/html/poly_collection/an_efficient_polymorphic_data_st.html (visited on 2021-05-19).

[25] D. J. Armstrong, "The Quarks of Object-Oriented Development," *Communications of the ACM*, 2006-02. DOI: 10.1145/1113034.1113040.

[26] Unity Technologies. (2021-04-13). "Part 1: Understand data-oriented design," [Online]. Available: https://learn.unity.com/tutorial/part-1-understand-data-oriented-design (visited on 2021-05-13).

[27] Oracle Corporation. (2021). "What Is an Object?" [Online]. Available: https://docs.oracle.com/javase/tutorial/java/concepts/object.html (visited on 2021-05-04).

[28] T. Albrecht, "Pitfalls of Object Oriented Programming," *Proceedings of Game Connect: Asia Pacific (GCAP)*, 2009.

[29] N. Llopis. (2009-12-04). "Data-Oriented Design (Or Why You Might Be Shooting Yourself in The Foot With OOP)," [Online]. Available: https://gamesfromwithin.com/data-oriented-design (visited on 2021-05-13).

[30] J. Linietsky and A. Manzur. (2021). "Godot's design philosophy," [Online]. Available: https://docs.godotengine.org/en/stable/getting_started/step_by_step/godot_design_philosophy.html (visited on 2021-05-24).

[31] The OGRE Team. "OGRE: Features," [Online]. Available: https://www.ogre3d.org/about/features (visited on 2021-05-24).

[32] Epic Games, Inc. (2021). "Unreal Engine 4 Documentation - Unreal Architecture," [Online]. Available: https://docs.unrealengine.com/en-US/ProgrammingAndScripting/ProgrammingWithCPP/UnrealArchitecture/index.html (visited on 2021-05-24).

[33] G. T. Delarosa. (2010-07-09). "Using Abstraction to Optimize Runtime Polymorphism," [Online]. Available: https://www.gamedev.net/tutorials/programming/general-and-gameplay-programming/using-abstraction-to-optimize-runtime-polymorphism-r2763/ (visited on 2021-05-13).

[34] E. Lengyel, *Game Engine Gems 2*, 2nd ed. India: A K Peters, 2011.

[35] Epic Games, Inc. (2021). "Unreal Engine 4 Documentation: AActor," [Online]. Available: https://docs.unrealengine.com/en-US/API/Runtime/Engine/GameFramework/AActor/index.html (visited on 2021-05-24).

[36] Unity Technologies. (2021-02-02). "Part 2: Data design," [Online]. Available: https://learn.unity.com/tutorial/part-2-data-design (visited on 2021-05-13).

[37] S. Nikolov, *Data-oriented design in practice*, Paper presented at the Meeting C++ 2018 conference, Berlin, 2018. [Online]. Available: https://meetingcpp.com/mcpp/slides/2018/ (visited on 2021-05-13).

[38] R. Nystrom, *Game Programming Patterns*, 1st ed. US: Genever Benning, 2014.

[39] A. Rodham. (2020-12-04). "Performance at scale: Sequencer in Unreal Engine 4.26," Epic Games, Inc., [Online]. Available: https://www.unrealengine.com/en-US/tech-blog/performance-at-scale-sequencer-in-unreal-engine-4-26 (visited on 2021-05-28).

[40] L. Meijer. (2019-03-08). "On DOTS: Entity Component System," Unity Technologies, [Online]. Available: https://blog.unity.com/technology/on-dots-entity-component-system (visited on 2021-05-30).

[41] C. Ferreira and M. Geig. (2018-05-31). "Get Started with the Unity Entity Component System (ECS), C# Job System, and Burst Compiler," Intel Corporation, [Online]. Available: https://software.intel.com/content/www/us/en/develop/articles/get-started-with-the-unity-entity-component-system-ecs-c-sharp-job-system-and-burst-compiler.html (visited on 2021-05-31).

[42] N. Savas. (2016-05-31). "Nomad Game Engine: Part 2 — ECS," [Online]. Available: https://medium.com/@savas/nomad-game-engine-part-2-ecs-9132829188e5 (visited on 2021-05-31).

[43] W. Faryabi, "Data-oriented design approach for processor intensive games," M.S. thesis, Norwegian University of Science and Technology, 2018.

[44] D. Nilsson and A. Björkman, "Profiling a Prototype Game Engine Based on an Entity Component System in C++," B.S. thesis, Malmö University, 2019.

[45] G. Löfstedt, "Design av komponentbaserade entitetssystem," M.S. thesis, University of Skövde, 2012.

[46] Unity Technologies. (2017-05-18). "Unity Manual - Time," [Online]. Available: https://docs.unity3d.com/Manual/class-TimeManager.html (visited on 2021-06-03).

[47] Microsoft Corporation. (2016-04-11). "Docs - /STACK (Stack Allocations)," [Online]. Available: https://docs.microsoft.com/en-us/cpp/build/reference/stack-stack-allocations?view=msvc-160 (visited on 2021-05-26).

[48] Intel Corporation. (2021). "Instrumentation and Tracing Technology APIs," [Online]. Available: https://software.intel.com/content/www/us/en/develop/documentation/vtune-help/top/api-support/instrumentation-and-tracing-technology-apis.html (visited on 2021-05-13).

[49] Microsoft Corporation. (2016-04-11). "Auto-Parallelization and Auto-Vectorization," [Online]. Available: https://docs.microsoft.com/en-us/cpp/parallel/auto-parallelization-and-auto-vectorization?view=msvc-160 (visited on 2021-05-26).

[50] Intel Corporation. (2021-05-10). "Top-down Microarchitecture Analysis Method," [Online]. Available: https://software.intel.com/content/www/us/en/develop/documentation/vtune-cookbook/top/methodologies/top-down-microarchitecture-analysis-method.html (visited on 2021-05-26).

[51] V. Tsymbal. (2021-04-01). "Intel® VTune™ Profiler System Requirements," Intel Corporation, [Online]. Available: https://software.intel.com/content/

www/us/en/develop/articles/vtune-profiler-system-requirements.html (visited on 2021-06-01).

[52] Intel Corporation. "Events for Intel® microarchitecture code name Haswell," [Online]. Available: https://perfmon-events.intel.com/haswell.html (visited on 2021-05-26).

[53] Japan Advanced Institute of Science and Technology. "L1 Data Cache Miss Rate," [Online]. Available: http://www.jaist.ac.jp/iscenter-new/mpc/altix/altixdata/opt/intel/vtune/doc/users_guide/mergedProjects/analyzer_ec/mergedProjects/reference_olh/pmp/ratios/l1_data_cache_miss_rate.html (visited on 2021-05-26).

[54] Japan Advanced Institute of Science and Technology. "L2 Cache Miss Rate," [Online]. Available: https://www.jaist.ac.jp/iscenter-new/mpc/altix/altixdata/opt/intel/vtune/doc/users_guide/mergedProjects/analyzer_ec/mergedProjects/reference_olh/pmp/ratios/l2_cache_miss_rate.html (visited on 2021-05-26).

[55] H. Brais. (2019-03-26). "An Introduction to the Cache Hit and Miss Performance Monitoring Events," [Online]. Available: https://hadibrais.wordpress.com/2019/03/26/an-introduction-to-the-cache-hit-and-miss-performance-monitoring-events/ (visited on 2021-05-26).

[56] Intel Corporation. (2021). "Intel® Core™ i7-4770K Processor," [Online]. Available: https://ark.intel.com/content/www/us/en/ark/products/75123/intel-core-i7-4770k-processor-8m-cache-up-to-3-90-ghz.html (visited on 2021-05-25).

[57] Intel Corporation. "Intel® Turbo Boost Technology 2.0," [Online]. Available: https://www.intel.com/content/www/us/en/architecture-and-technology/turbo-boost/turbo-boost-technology.html (visited on 2021-05-26).

[58] Intel Corporation. (2019-05-06). "Understanding How General Exploration Works in Intel® VTune™ Amplifier," [Online]. Available: https://software.intel.com/content/www/us/en/develop/articles/understanding-how-general-exploration-works-in-intel-vtune-amplifier-xe.html (visited on 2021-06-01).

[59] J. Franco, M. Hagelin, T. Wrigstad, S. Drossopoulou, and S. Eisenbach, "You can have it all: Abstraction and good cache performance," 2017-10, pp. 148–167. DOI: 10.1145/3133850.3133861.